UNIVERSIDAD AUTÓNOMA “GABRIEL RENÉ MORENO”

FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

PRIMER PARCIAL

“SOFTWARE DE ASISTENCIA CON ARQUITECTURA DE 3 CAPAS”

MATERIA	:	ARQUITECTURA DE SOFTWARE
DOCENTE	:	ING. JOSUE OBED VEIZAGA GONZALES
FECHA	:	20 / 04 / 2026
ESTUDIANTE	:	WILBERT ALEXANDER INTURIAS BANEGAS
REGISTRO	:	217169651

SANTA CRUZ – BOLIVIA

Tabla de contenido

1 CAPTURA DE REQUISITOS 1
1.1 Requisitos funcionales: 1
1.1.1 Identificar Actores y Casos de uso 1
1.1.2 Detalles de caso de uso 2
1.2 Requisitos no funcionales 16
1.2.1 Rendimiento 16
1.2.2 Usabilidad 16
1.2.3 Escalabilidad 16
1.2.4 Compatibilidad 16
1.2.5 Mantenimiento 16
1.2.6 Disponibilidad 16
1.2.7 Portabilidad 16
2 ANALISIS 17
2.1 Identificación de los módulos 17
2.1.1 Modulo Administración 17
2.1.2 Modulo Asistencia 17
2.2 Vista de módulos 18
2.2.1 Modulo Administración 18
2.2.2 Modulo Asistencia 18
3 DISEÑO 19
3.1 Diseño de la Arquitectura Lógica 19
3.1.1 Administración 19
3.1.2 Asistencia 19
3.2 Diseño de la base de datos 20

3.2.1 Diseño conceptual 20
3.2.2 Diseño Lógico 21
3.2.3 Diseño Físico 22
3.3 Diseño de Detalle Procedimental 24
3.3.1 Diagrama de clases dinámico 24
3.3.1 Diagrama de secuencia 31
3.4 Diseño de la interfaz 37
3.4.1 CU01: Gestionar Estudiantes 37
3.4.2 CU02: Gestionar Materias 37
3.4.3 CU03: Gestionar Grupo 37
3.4.4 CU04: Gestionar Horario 38
3.4.5 CU05: Registrar Asistencia QR 38
3.4.6 CU06: Gestionar Aula 38
3.4.7 CU07: Gestionar Inscripción 39
4 Bibliografia 40

1 CAPTURA DE REQUISITOS

1.1 Requisitos funcionales:

1.1.1 Identificar Actores y Casos de uso

Identificar Actores

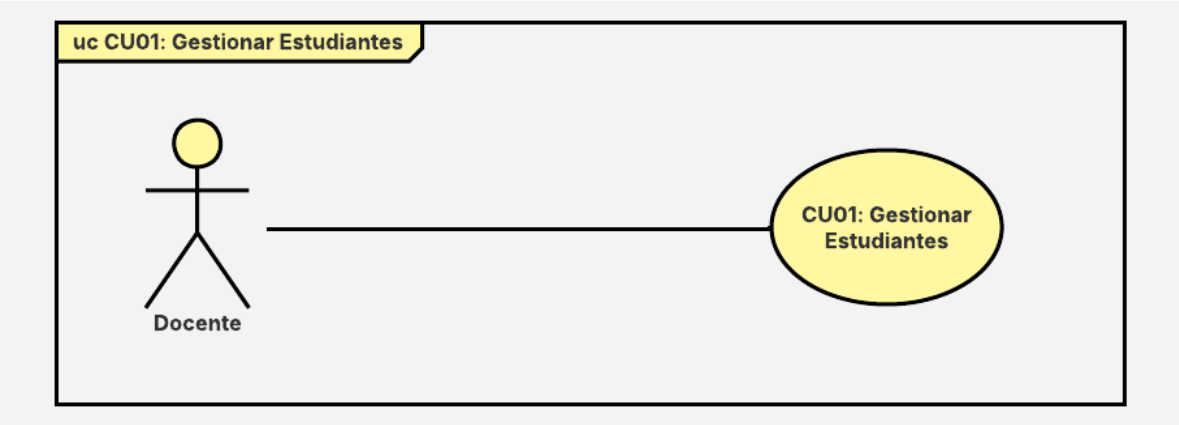
- Docente: Es el encargado de la gestión académica. Tiene permisos para gestionar materias, grupos, horarios e inscribir a los estudiantes en sus respectivos grupos. También es quien genera el QR para la asistencia.
- Estudiante: Es el beneficiario del sistema. Su interacción principal consiste en registrar su asistencia mediante el escaneo del código QR generado por el docente.

Identificar Casos de Uso

- CU01: Gestionar Estudiantes
- CU02: Gestionar Materias
- CU03: Gestionar Grupos
- CU04: Gestionar Horario
- CU05: Registrar Asistencia QR
- CU06: Gestionar Aula
- CU07: Gestionar Inscripción

1.1.2 Detalles de caso de uso

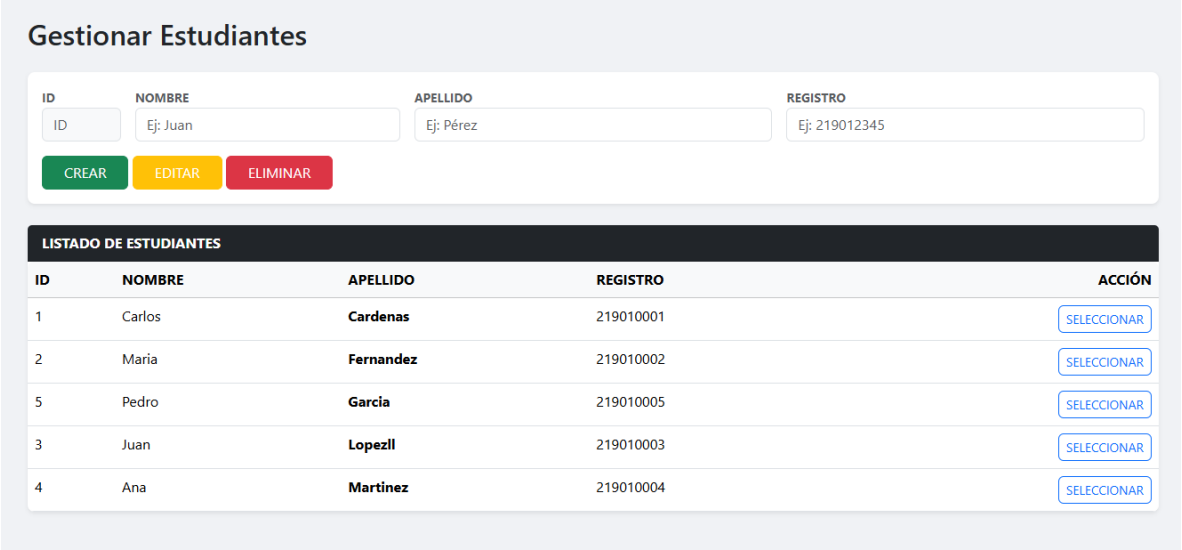
1.1.2.1 CU01: Gestionar Estudiantes



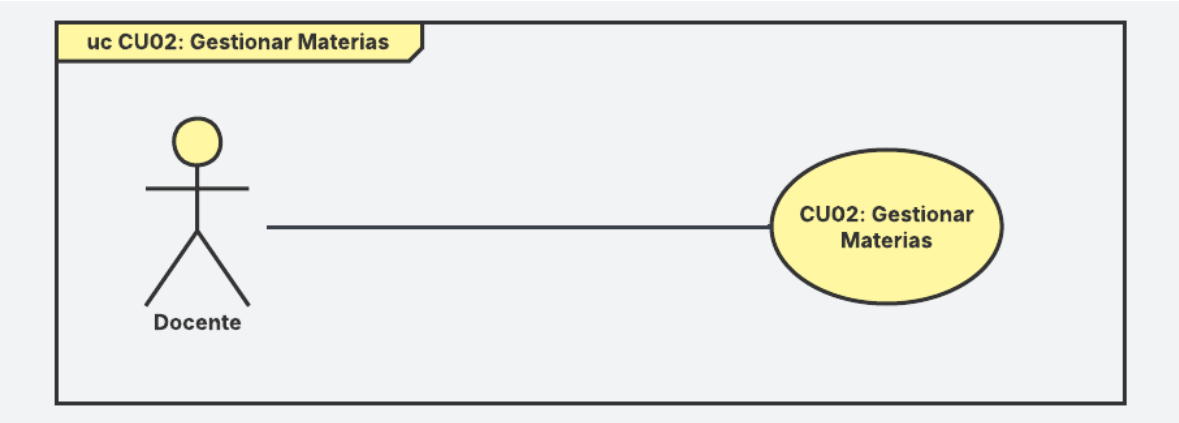
Nombre de caso de uso	CU01: Gestionar Estudiantes
Propósito	Gestionar los registros de Estudiantes
Descripción	Permite registrar, modificar y eliminar Estudiantes registrados como también ver la lista de asistencia de cada uno
Actores	Docente
Actor iniciador	Docente
Pre condición	Ninguna
Flujo principal	**Registrar Estudiante** 1. El Docente selecciona **Registrar estudiante**. 2. El sistema muestra formulario: *nombre, apellido, correo_electrónico, contraseña*. 3. El Docente completa los campos obligatorios. 4. El sistema valida: campos, formato. 5. El sistema crea el Usuario. 6. El sistema actualiza y muestra la lista de estudiantes del Docente. **Modificar Estudiante** 1. El Docente selecciona un estudiante. 2. El sistema muestra los datos actuales. 3. El Docente edita nombre, apellido, correo. 4. El sistema valida y guarda cambios (actualiza **Usuario**). 5. Se actualiza la lista. **Eliminar Estudiante** 1. El Docente selecciona Eliminar Estudiante. 2. El sistema solicita confirmación.

	3. El sistema aplica la acción y actualiza la lista. **Listar Estudiante** 1. El Docente desde el menú principal selecciona Estudiante. 2. El sistema muestra una presentación de los estudiantes completa
Postcondición	El estudiante queda registrado, modificado o eliminado según corresponda. Las listas de estudiantes del Docente reflejan los cambios.
Excepción	Correo ya existente. Formato de correo inválido. Intento de guardar con campos obligatorios vacíos.

1.1.2.1.1 Prototipos



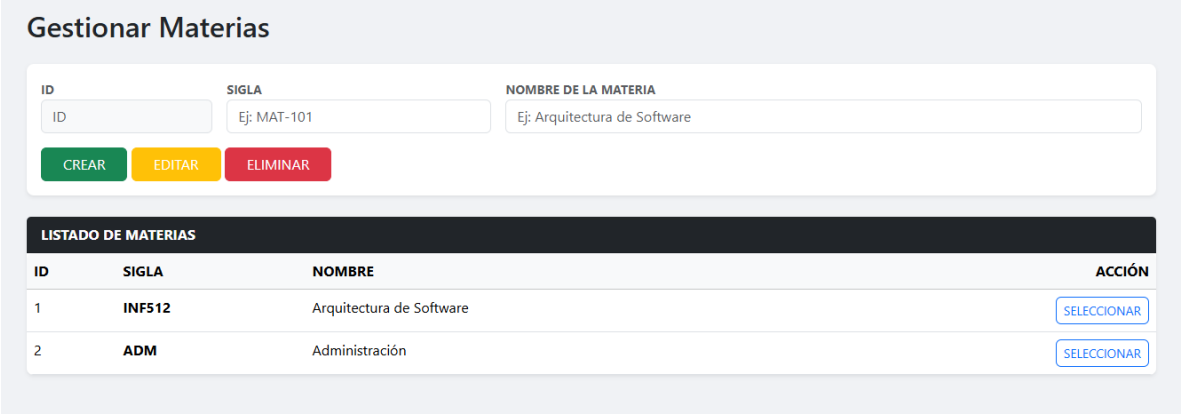
1.1.2.2 CU02: Gestionar Materias



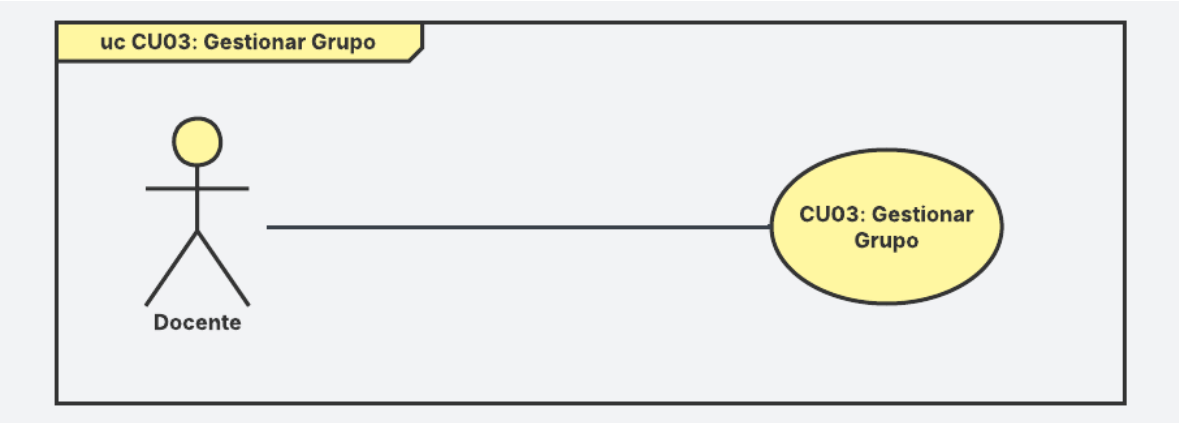
Nombre de caso de uso	CU02: Gestionar Materias
Propósito	Permitir al **Docente** crear, modificar y eliminar materias del sistema.
Descripción	Permite al docente Gestionar sus **Materias** sigla, nombre materia, nivel.
Actores	Docente
Actor iniciador	Docente
Pre condición	Ninguna.
Flujo principal	**Registrar Materia** El Docente selecciona **Registrar materia**. El sistema muestra formulario con: **sigla**, **nombre materia**, **nivel**. El Docente completa los campos obligatorios. El sistema valida: Campos obligatorios llenos. El sistema guarda la **Materia** y muestra la lista actualizada. **Modificar Materia** El Docente elige una materia de la lista. El sistema muestra sus datos actuales. El Docente edita sigla, nombre materia y/o nivel. El sistema valida. El sistema guarda y actualiza la lista. Los **Grupos** asociados permanecen vinculados a la materia editada. **Eliminar Materia** El Docente selecciona **Eliminar** sobre una materia. El sistema solicita confirmación.

	El sistema verifica relaciones **Si no existen relaciones**, procede con la eliminación. El sistema aplica la acción y actualiza la lista. **Listar Materia** El Docente desde el menú principal selecciona Materia. El sistema muestra una presentación de las materias completa
Postcondición	La materia queda creada, modificada, eliminada o desactivada correctamente. La lista de materias refleja los cambios.
Excepción	sigla duplicada. Intento de guardar con campos obligatorios vacíos.

1.1.2.2.1 Prototipos



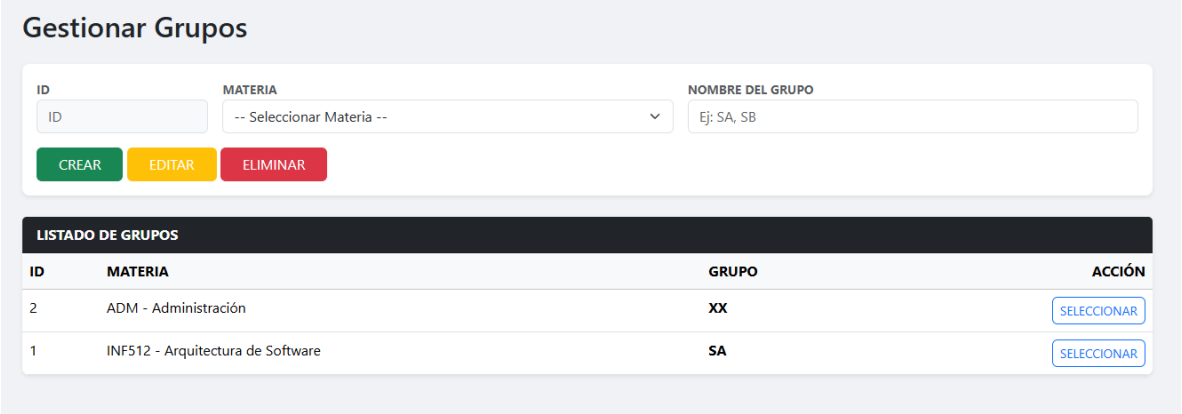
1.1.2.3 CU03: Gestionar Grupo



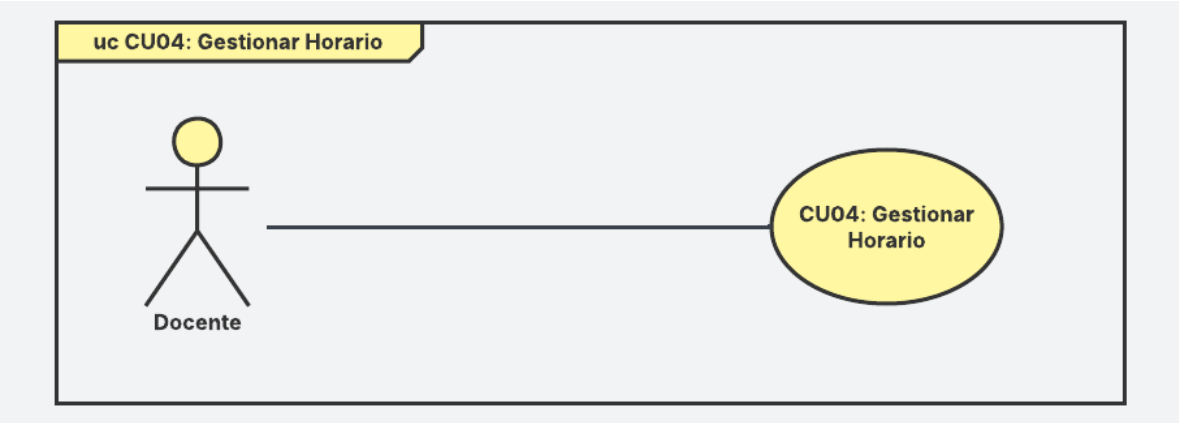
Nombre de caso de uso	CU03: Gestionar Grupo
Propósito	Permitir al **Docente** crear, modificar y eliminar **grupos** en el sistema como también asignar o quitar estudiante de grupos.
Descripción	El Docente gestiona sus **Grupos** pudiendo crear, editar, eliminar, listar además de asignar y quitar estudiantes del grupo.
Actores	Docente
Actor iniciador	Docente
Pre condición	1. Debe existir al menos una **Materia** registrada para asociar el grupo.
Flujo principal	**Registrar grupo** El Docente selecciona **Registrar grupo**. El sistema muestra formulario con: **nombre** *(obligatorio)*, **materia**. El Docente completa los campos. El sistema **valida**: campos obligatorios, nombre no vacío. El sistema **guarda** el Grupo y muestra la **lista actualizada**. **Modificar grupo** El Docente elige un grupo de la lista. El sistema muestra sus datos actuales. El Docente edita **nombre** y/o **materia** (si aplica). El sistema valida (campos, duplicados en la misma materia). El sistema **actualiza** el Grupo y muestra la lista. **Eliminar grupo** El Docente selecciona **Eliminar** sobre un grupo.

	El sistema solicita **confirmación**. El sistema **verifica relaciones** El sistema **elimina** **Listar Grupo** El Docente desde el menú principal selecciona Materia. El sistema muestra una presentación de los grupos completa **Asignar Estudiante a grupo** Selecciona un grupo. En el formulario de asignación elegir un estudiante del select y la fecha de unión Dar click en “asignar” **Quitar Estudiante a grupo** Selecciona un grupo. Visualizar la tabla de estudiantes asignados Dar click en “quitar”
Postcondición	El grupo queda **creado, modificado, eliminado** correctamente. La **lista de grupos** refleja los cambios.
Excepción	**Nombre duplicado** en la misma materia. Intento de guardar con **campos obligatorios** vacíos.

1.1.2.3.1 Prototipos



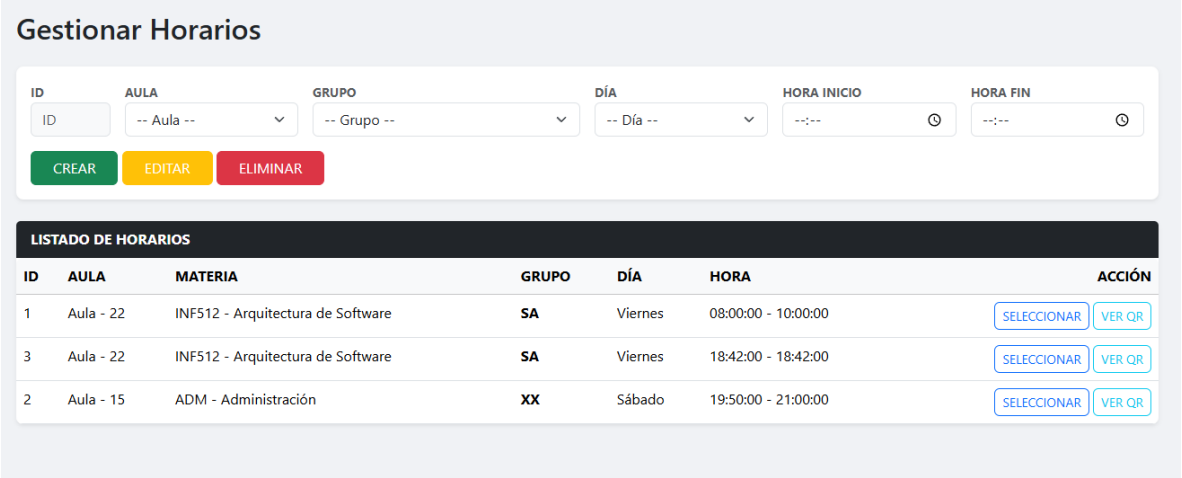
1.1.2.4 CU04: Gestionar Horario



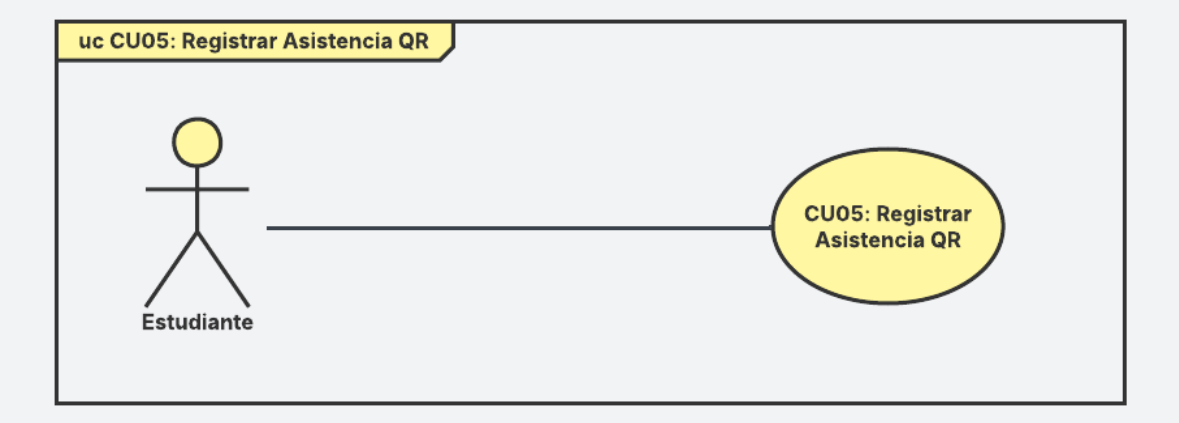
Nombre de caso de uso	CU04: Gestionar Horario
Propósito	Permitir al **Docente** crear, modificar y eliminar **horarios** de clase asociados a un grupo y a un aula.
Descripción	El docente gestiona los horarios indicando grupo, aula, día de la semana y horas de inicio/fin. Esto define cuándo y dónde se dictan las materias y permite posteriormente controlar la asistencia.
Actores	Docente
Actor iniciador	Docente
Pre condición	1. Debe existir al menos un **Grupo** y un **Aula** registrados.
Flujo principal	**Registrar Horario** El Docente selecciona **Registrar horario**. El sistema muestra formulario con: **grupo**, **aula**, **día de semana**, **hora inicio**, **hora fin**. El Docente completa los campos. El sistema valida: Campos obligatorios llenos. **Rangos horarios válidos** (hora inicio < hora fin). El sistema guarda el **Horario** y muestra la lista actualizada. **Modificar Horario** El Docente selecciona un horario existente de la lista. El sistema muestra los datos actuales. El Docente edita: aula, día, hora inicio y/o hora fin. El sistema valida los cambios (campos obligatorios, rangos válidos, no solapamiento). El sistema guarda y muestra la lista de horarios actualizada.

	Eliminar horario El Docente selecciona **Eliminar** sobre un horario. El sistema solicita confirmación. El sistema verifica relaciones: Se procede con la eliminación. El sistema aplica la acción y actualiza la lista. **Listar Grupo** El Docente desde el menú principal selecciona Horario. El sistema muestra una presentación de los Horarios completa
Postcondición	El horario queda creado, modificado, eliminado correctamente. La lista de horarios refleja los cambios. Se mantiene la integridad: no hay choques de horarios ni registros huérfanos de asistencia.
Excepción	Campos obligatorios vacíos. Hora inicio ≥ hora fin.

1.1.2.4.1 Prototipos



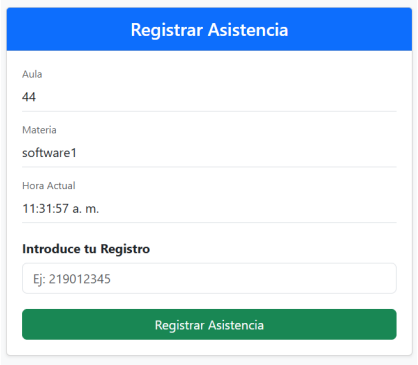
1.1.2.5 CU5: Registrar Asistencia QR



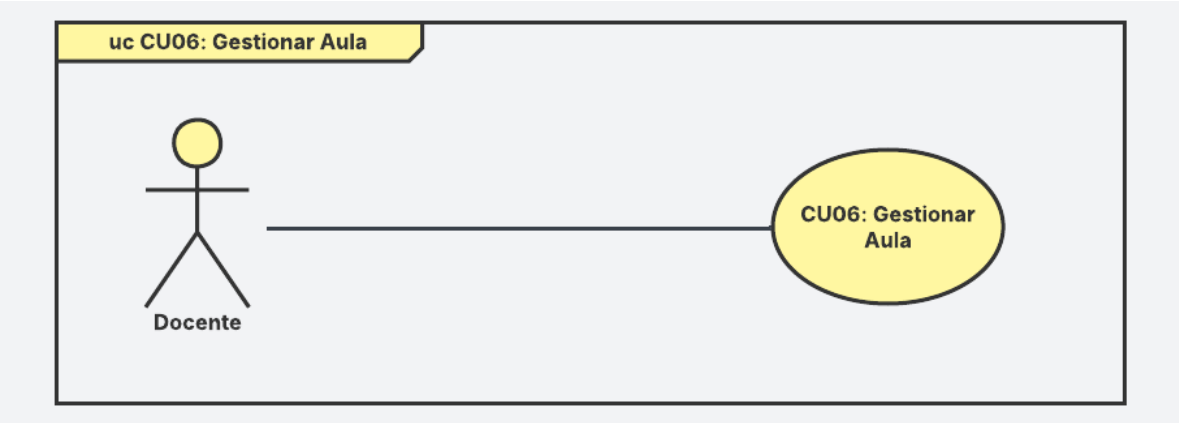
Nombre de caso de uso	CU5. Registrar asistencia QR
Propósito	Permitir al **Estudiante** registrar su asistencia mediante el escaneo de un código QR y relleno de un fomulario.
Descripción	El estudiante utiliza su dispositivo para escanear el **QR** asociado al aula/horario en curso. El sistema valida el código, la hora y la relación estudiante-grupo y registra la Asistencia.
Actores	Estudiante
Actor iniciador	Estudiante
Pre condición	1. El estudiante debe estar inscrito en el grupo (Inscripcion).
Flujo principal	**Registrar ministerio** El estudiante escanea el código qr que previamente a sido generado por el docente El estudiante verifica la información y coloca su numero de registro El estudiante selecciona la opción **Registrar asistencia**. El sistema valida que corresponda a un **aula y horario** vigentes, que la hora de registro esté dentro del rango válido (hora_inicio y hora_fin), que el estudiante esté inscrito en el grupo correspondiente. Si las validaciones son correctas, el sistema crea un registro en Asistencia con: Id de estudiante Id de horario Fecha y hora Estado de asistencia = presente

	El sistema confirma la asistencia registrada al estudiante.
Postcondición	La asistencia queda registrada en la tabla Asistencia. El estudiante recibe confirmación (ej. mensaje en pantalla). La asistencia puede ser consultada posteriormente por el docente.
Excepción	**Estudiante no inscrito**: El estudiante no se encuentra inscrito **Asistencia ya registrada**: el estudiante intenta registrar más de una vez en la misma sesión.

1.1.2.5.1 Prototipos



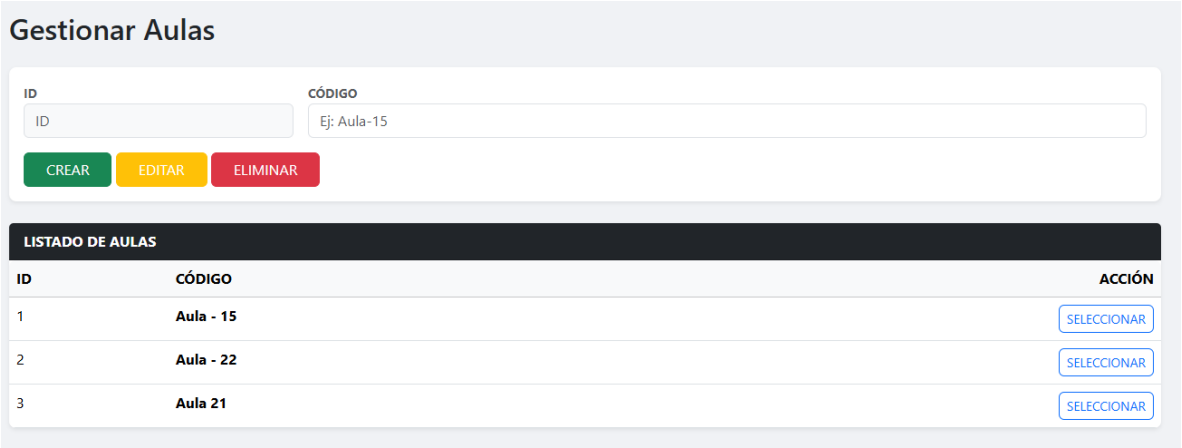
1.1.2.6 CU6: Gestionar Aula



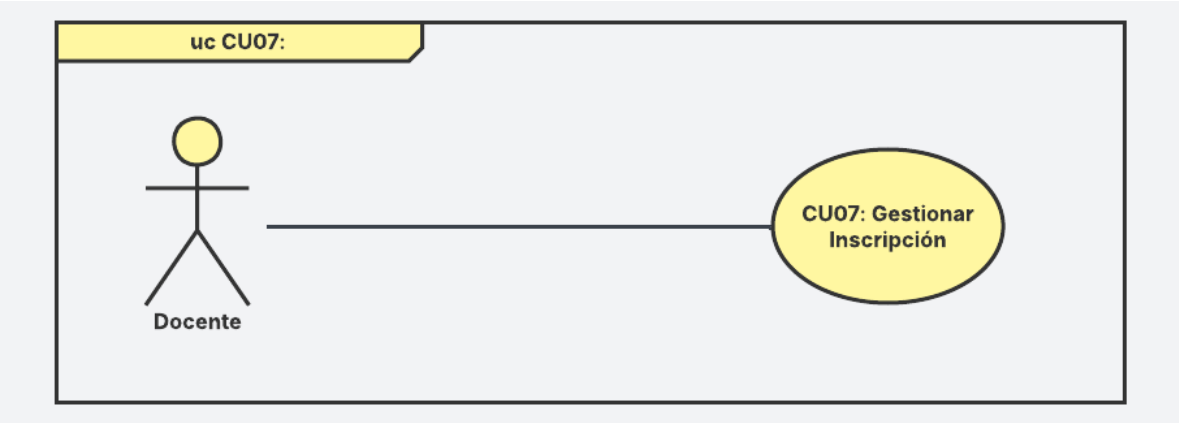
Nombre de caso de uso	CU6. Gestionar Aula
Propósito	Permitir al **Docente** crear, actualizar y eliminar **aulas**, así como **generar** el código **QR** que se utilizará para el registro de asistencia.
Descripción	El Docente administra las aulas del sistema. Cada aula posee un **código** único y un **QR** que puede generarse.
Actores	Docente
Actor iniciador	Docente
Pre condición	1. Ninguna.
Flujo principal	**Registrar Aula** El Docente selecciona **Registrar aula**. El sistema muestra formulario con: **código** (obligatorio, único) y **opción para generar QR** (opcional en el alta). El Docente completa los campos. El sistema **valida**: código no vacío y **no duplicado**. El sistema **guarda** el Aula **genera** y asocia qr_code. El sistema muestra la **lista de aulas** actualizada. **Modificar Aula** El Docente elige un aula de la lista. El sistema muestra los datos actuales. El Docente edita el **código** El sistema valida (código no vacío, **no duplicado**). El sistema **actualiza** el aula. El sistema muestra la lista actualizada. Los **Horarios** asociados **se mantienen** vinculados al aula. **Eliminar Aula**

	El Docente selecciona **Eliminar** sobre un aula. El sistema solicita **confirmación**. El sistema **elimina** el aula y actualiza la lista. **Generar QR de sesión** El Docente hace clic en **Generar QR** (o **Regenerar QR**). El sistema genera el código qr El sistema **asocia** el código qr al aula y lo **muestra** para que sea compartido.
Postcondición	El aula queda **creada, modificada, eliminada/desactivada** correctamente. La **lista de aulas** refleja los cambios. Si se generó QR, el aula queda con un código qr **vigente** y listo para ser utilizado en registro de asistencia.
Excepción	1. **Código duplicado** de aula. 2. Intento de guardar con **campos obligatorios** vacíos.

1.1.2.6.1 Prototipos



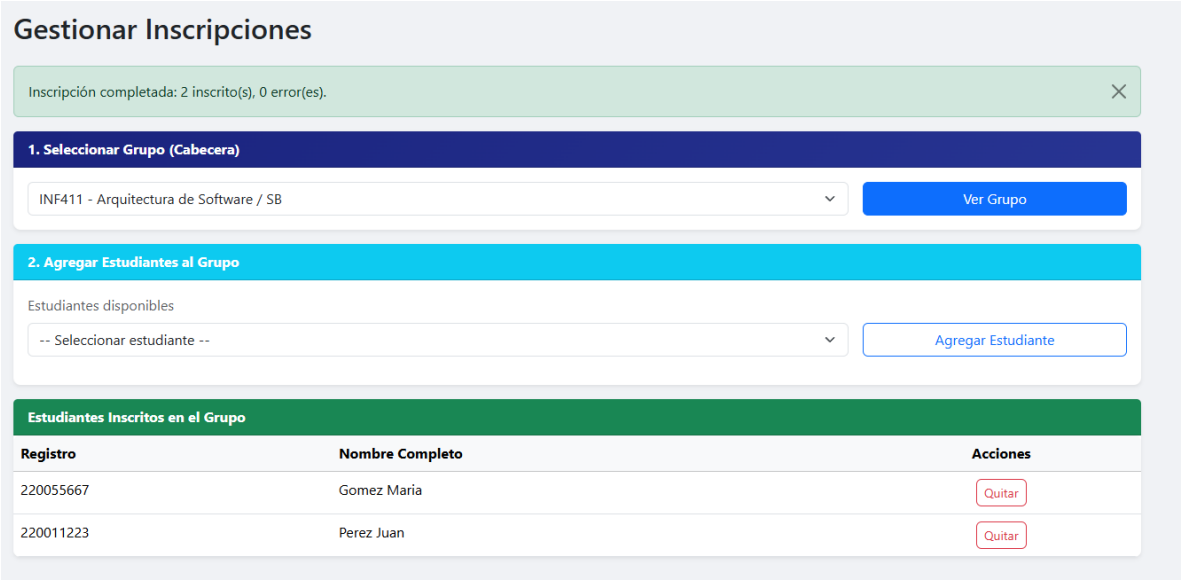
1.1.2.7 CU7: Gestionar Inscripción



Nombre de caso de uso	**CU07: Gestionar Inscripción**
Propósito	Gestionar la asignación masiva de estudiantes a grupos específicos para el control de asistencia.
Descripción	Permite al docente seleccionar un grupo de materia, conformar una lista temporal de estudiantes y realizar el registro transaccional en la base de datos. También permite remover estudiantes previamente inscritos.
Actores	Docente
Actor iniciador	Docente
Pre condición	1. Ninguna.
Flujo principal	**A. Seleccionar Grupo (Cabecera)** 1. El Docente selecciona un Grupo de la lista desplegable. 2. El sistema carga y muestra los estudiantes que ya pertenecen al grupo. 3. El sistema habilita el selector de estudiantes disponibles (no inscritos). **B. Conformar Lista Temporal (Detalle con JS)** 1. El Docente busca y selecciona un estudiante del combo de disponibles. 2. El Docente hace clic en el botón "Agregar". 3. El sistema añade al estudiante a una tabla temporal en la vista (sin guardar en BD aún). **C. Registrar Inscripción (Transaccional - loop foreach)** 1. El Docente revisa la lista temporal y hace clic en "Registrar Inscripción". 2. El sistema crea un registro de cabecera en la tabla inscripcion.

	3. El sistema recorre la lista temporal (foreach) e inserta cada estudiante en detalle_inscripcion. 4. El sistema limpia la lista temporal y actualiza la tabla general de inscritos. **D. Quitar Estudiante** 1. El Docente identifica un estudiante en la tabla de inscritos. 2. El Docente hace clic en la opción "Quitar". 3. El sistema solicita confirmación al usuario. 4. El sistema elimina el registro del detalle y actualiza la vista.
Postcondición	Los estudiantes quedan vinculados al grupo y habilitados para marcar asistencia en los horarios correspondientes.
Excepción	1. Intentar registrar una inscripción con la lista temporal vacía. 2. Error de conexión con la base de datos durante el proceso transaccional. 3. Registro de estudiante ya existente en el grupo (controlado por el combo de disponibles).

1.1.2.7.1 Prototipos



1.2 Requisitos no funcionales

1.2.1 Rendimiento

- El sistema debe cargar cada módulo en menos de 3 segundos.
- Las operaciones (insertar, editar, eliminar) deben reflejarse inmediatamente.

1.2.2 Usabilidad

- La interfaz debe ser clara, intuitiva y fácil de usar.

1.2.3 Escalabilidad

- Debe permitir aumentar usuarios y módulos sin rediseñar la arquitectura.
- Soportar al menos 500 registros por entidad sin pérdida de rendimiento.

1.2.4 Compatibilidad

- Funcionar en navegadores modernos (Chrome, Firefox, Edge).
- Compatible con Windows, Linux y macOS.

1.2.5 Mantenimiento

- Seguir arquitectura de 3 capas.
- Código organizado por módulos y responsabilidades.

1.2.6 Disponibilidad

- Disponible al menos el 95% del tiempo.
- Acceso 24/7 desde navegador.

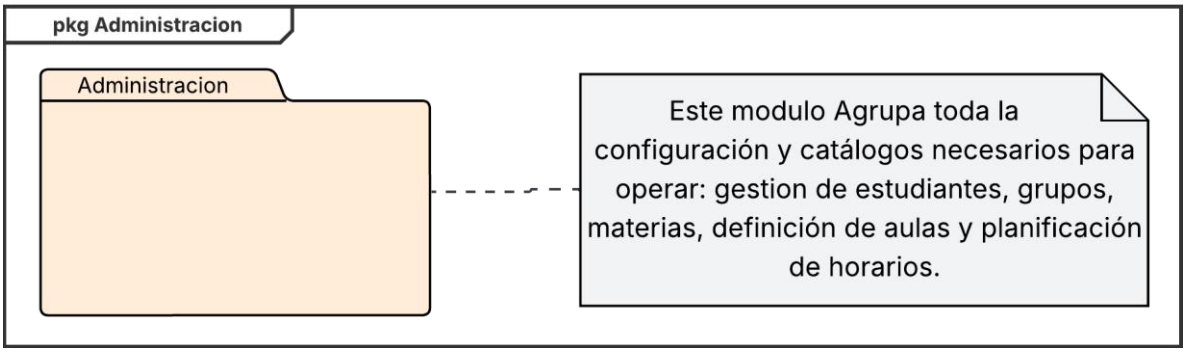
1.2.7 Portabilidad

- Desplegable en servidores locales o en la nube.

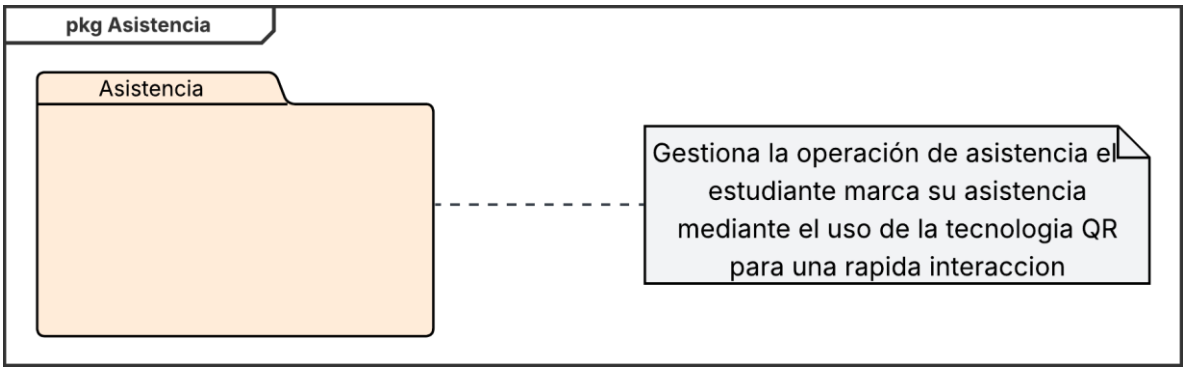
2 ANALISIS

2.1 Identificación de los módulos

2.1.1 Modulo Administración

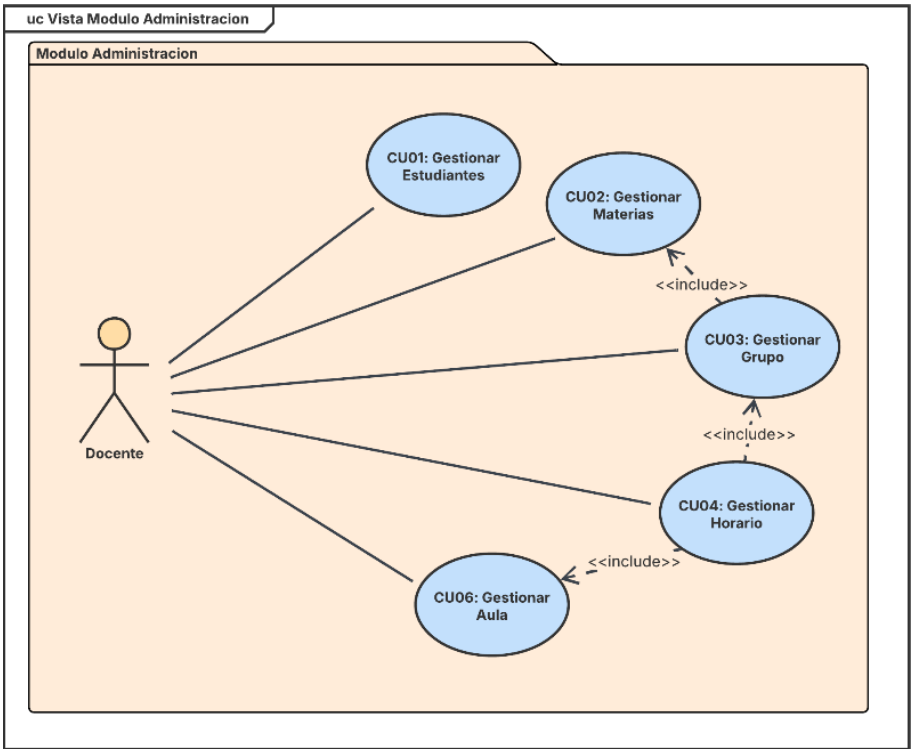


2.1.2 Modulo Asistencia

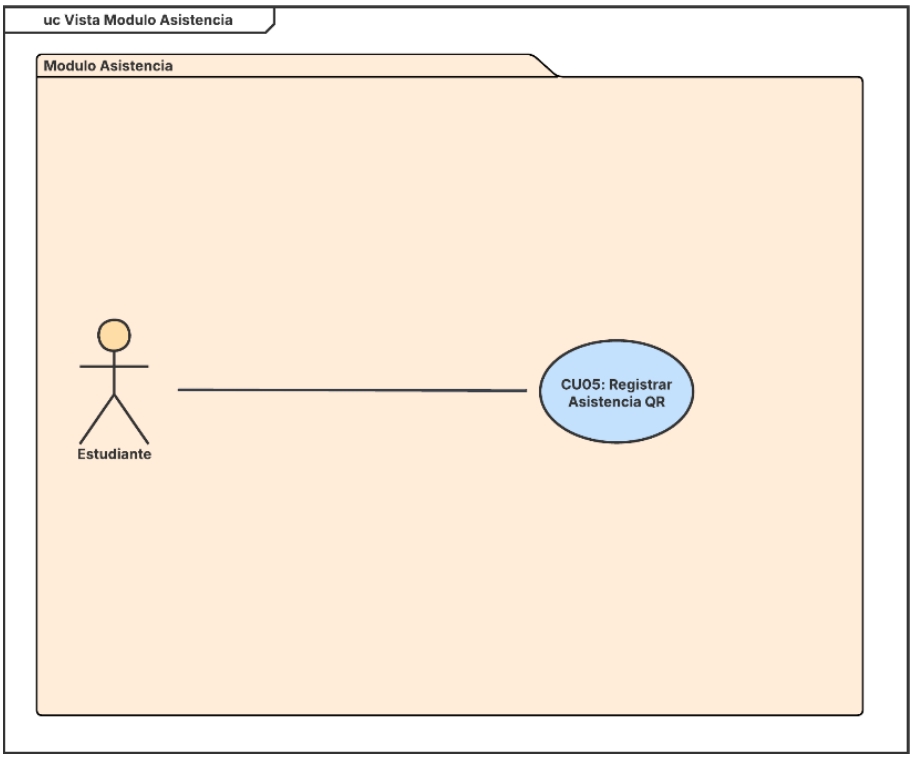


2.2 Vista de módulos

2.2.1 Modulo Administración



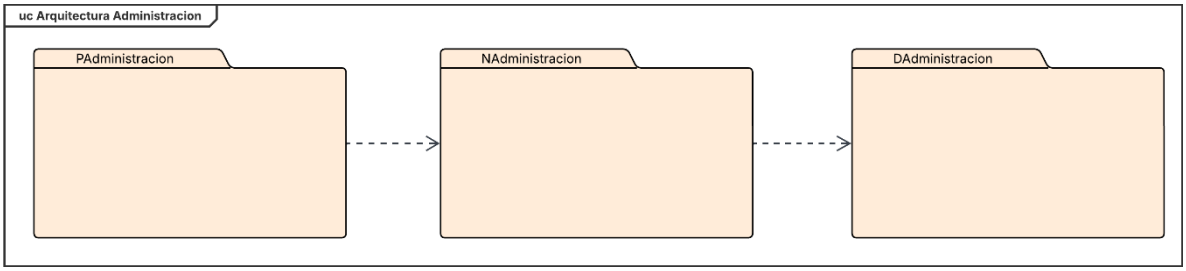
2.2.2 Modulo Asistencia



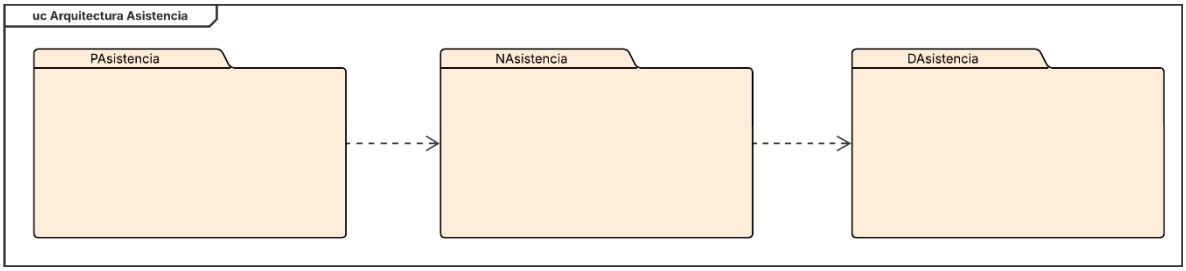
3 DISEÑO

3.1 Diseño de la Arquitectura Lógica

3.1.1 Administración

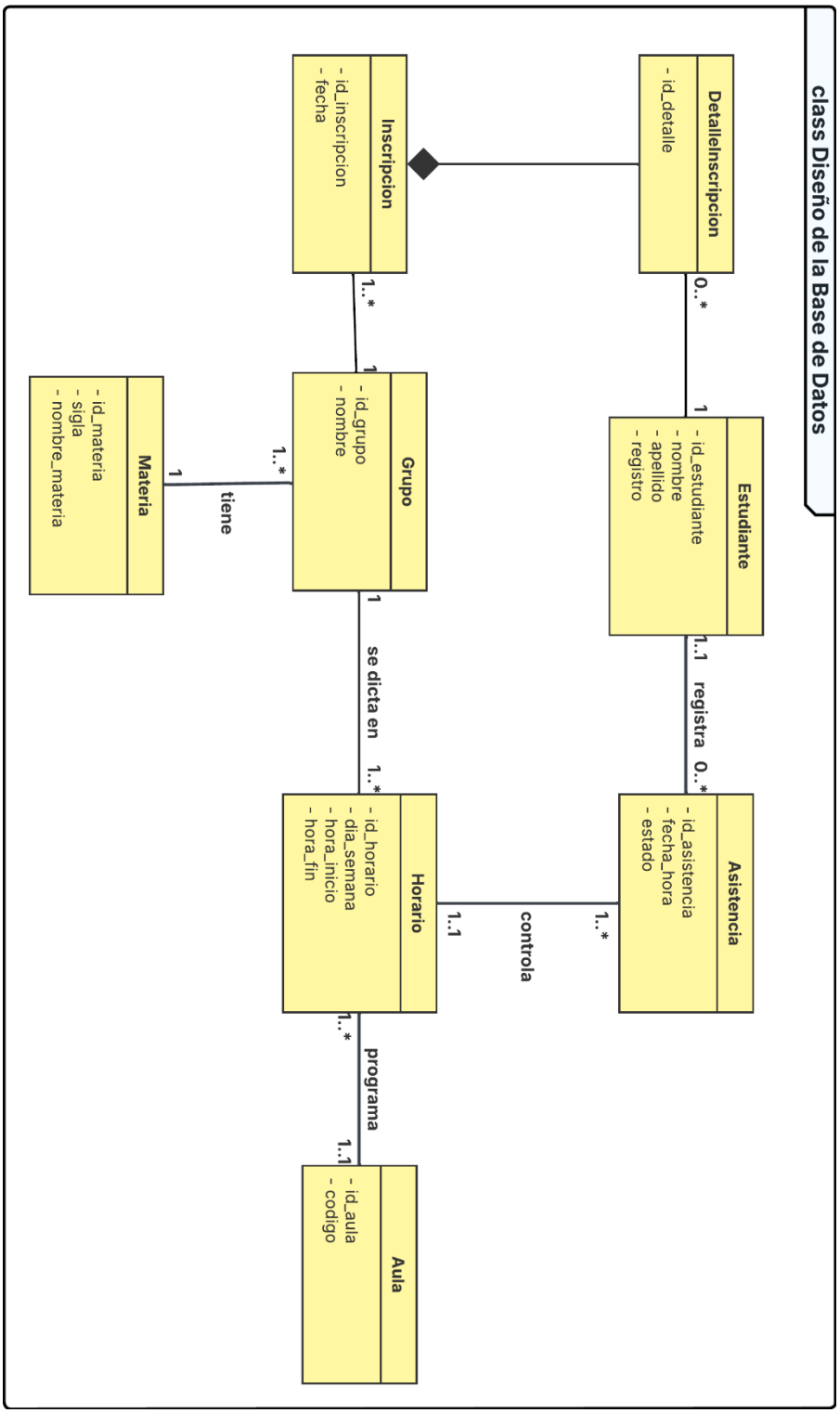


3.1.2 Asistencia



3.2 Diseño de la base de datos

3.2.1 Diseño conceptual



3.2.2 Diseño Lógico

3.2.2.1 Tabla Usuario

PK						
Id_usuario	nombre	apellido	correo	registro	tipo_usuario	password

3.2.2.2 Tabla Materia

PK			
id_materia	sigla	nombre_materia	nivel

3.2.2.3 Tabla Grupo

PK		**FK**
id_grupo	nombre	id_materia

3.2.2.4 Tabla Aula

PK		**FK**
id_aula	codigo	qr_code

3.2.2.5 Tabla horario

PK				**FK**	**FK**
id_horario	dia_semana	hora_inicio	hora_fin	**id_aula**	**id_grupo**

3.2.2.6 Tabla Inscripción

PK/FK	**PK/FK**	
id_estudiante	**id_grupo**	fecha_inscripcion

3.2.2.7 Tabla Asistencia

PK			**FK**	**FK**
id_asistencia	fecha_hora	estado_asistencia	**id_estudiante**	**id_horario**

3.2.3 Diseño Físico

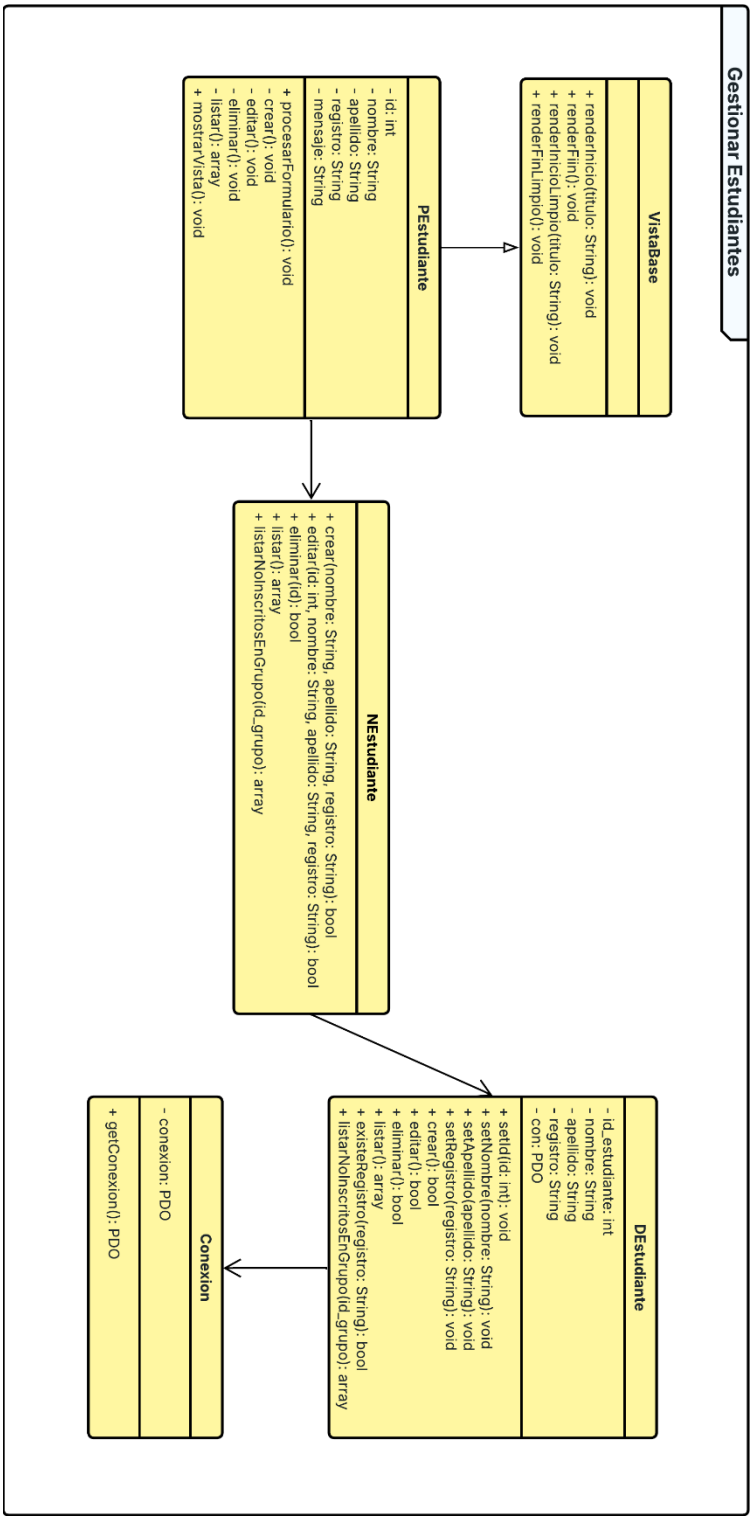
```
CREATE TABLE estudiante (
id_usuario SERIAL PRIMARY KEY,
nombre VARCHAR(80) NOT NULL,
apellido VARCHAR(80) NOT NULL,
correo VARCHAR(160) NOT NULL UNIQUE,
registro VARCHAR(40),
tipo_usuario VARCHAR(20) NOT NULL,
password TEXT NOT NULL
);
CREATE TABLE materia (
id_materia SERIAL PRIMARY KEY,
sigla VARCHAR(20) NOT NULL UNIQUE,
nombre_materia VARCHAR(120) NOT NULL,
nivel VARCHAR(20) NOT NULL
);
CREATE TABLE grupo (
id_grupo SERIAL PRIMARY KEY,
id_materia INT NOT NULL,
nombre VARCHAR(80) NOT NULL,
CONSTRAINT grupo_unico_por_materia UNIQUE (id_materia, nombre),
FOREIGN KEY (id_materia) REFERENCES materia(id_materia) ON DELETE
RESTRICT
);
CREATE TABLE aula (
id_aula SERIAL PRIMARY KEY,
codigo VARCHAR(40) NOT NULL UNIQUE,
qr_code TEXT
);
-- HORARIO
CREATE TABLE horario (
id_horario SERIAL PRIMARY KEY,
```

```
id_aula INT NOT NULL,
id_grupo INT NOT NULL,
dia_semana SMALLINT NOT NULL,
hora_inicio TIME NOT NULL,
hora_fin TIME NOT NULL,
FOREIGN KEY (id_aula) REFERENCES aula(id_aula) ON DELETE RESTRICT,
FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE CASCADE
);
CREATE TABLE inscripcion (
id_estudiante INT NOT NULL,
id_grupo INT NOT NULL,
fecha_inscripcion DATE NOT NULL DEFAULT CURRENT_DATE,
CONSTRAINT inscripcion_unica UNIQUE (id_estudiante, id_grupo),
PRIMARY KEY (id_estudiante, id_grupo)
FOREIGN KEY (id_estudiante) REFERENCES usuario(id_usuario) ON DELETE
CASCADE,
FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE CASCADE
);
CREATE TABLE asistencia (
id_asistencia SERIAL PRIMARY KEY,
id_estudiante INT NOT NULL,
id_horario INT NOT NULL,
fecha_hora TIMESTAMP NOT NULL DEFAULT NOW(),
estado_asistencia VARCHAR(15) NOT NULL DEFAULT 'presente',
FOREIGN KEY (id_estudiante) REFERENCES usuario(id_usuario) ON DELETE
CASCADE,
FOREIGN KEY (id_horario) REFERENCES horario(id_horario) ON DELETE
CASCADE
);
```

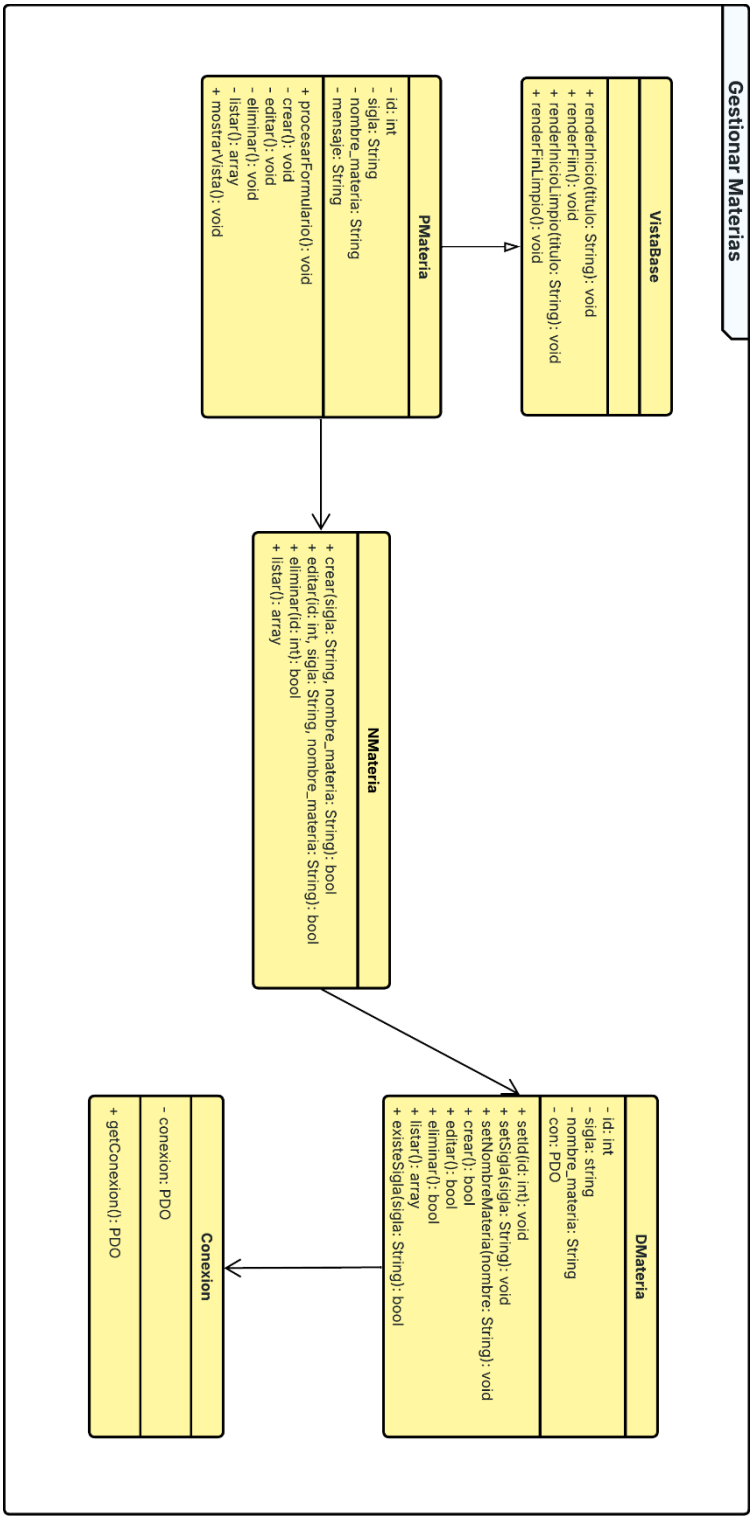
3.3 Diseño de Detalle Procedimental

3.3.1 Diagrama de clases dinámico

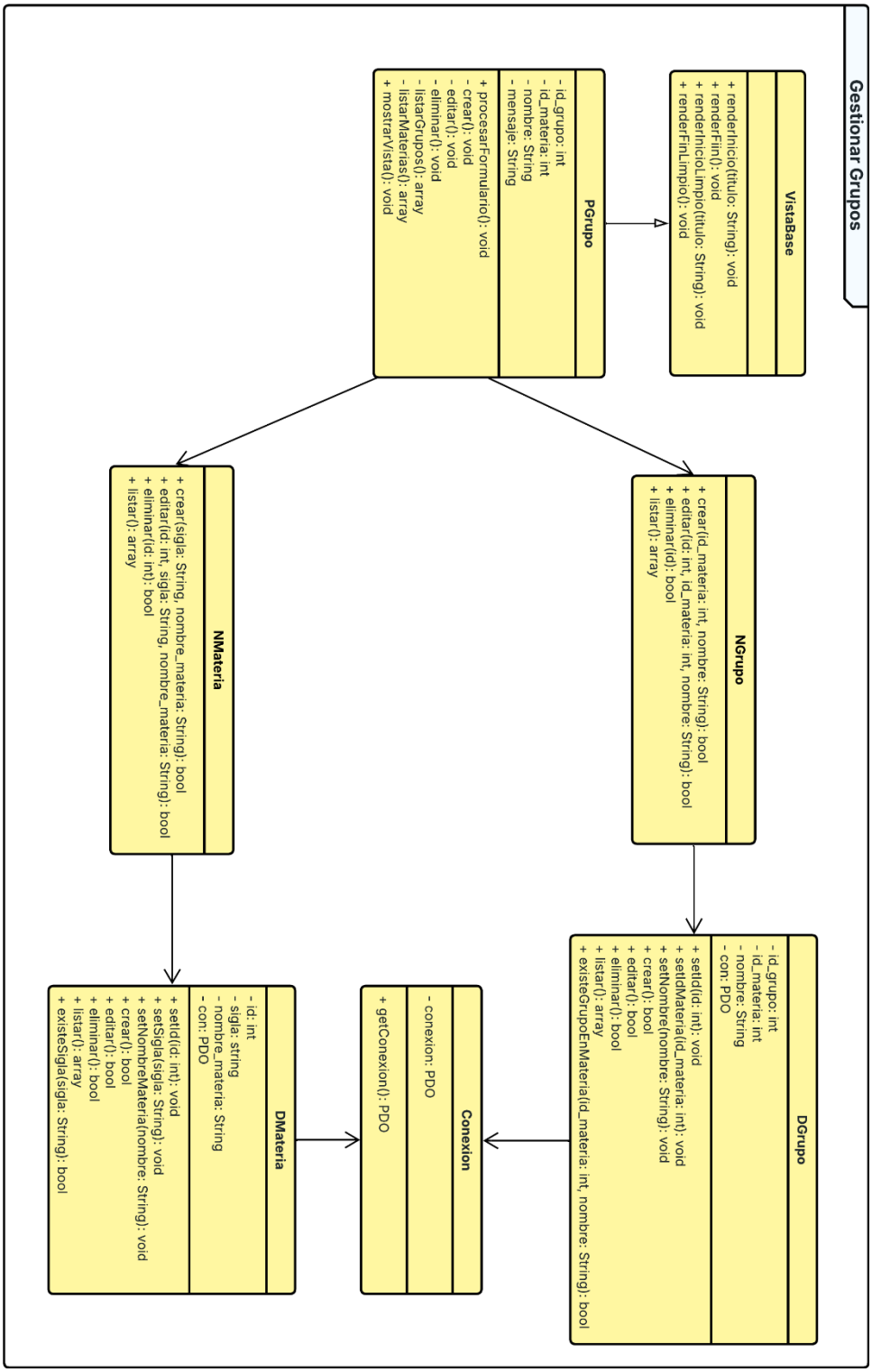
3.3.1.1 CU01: Gestionar Estudiantes



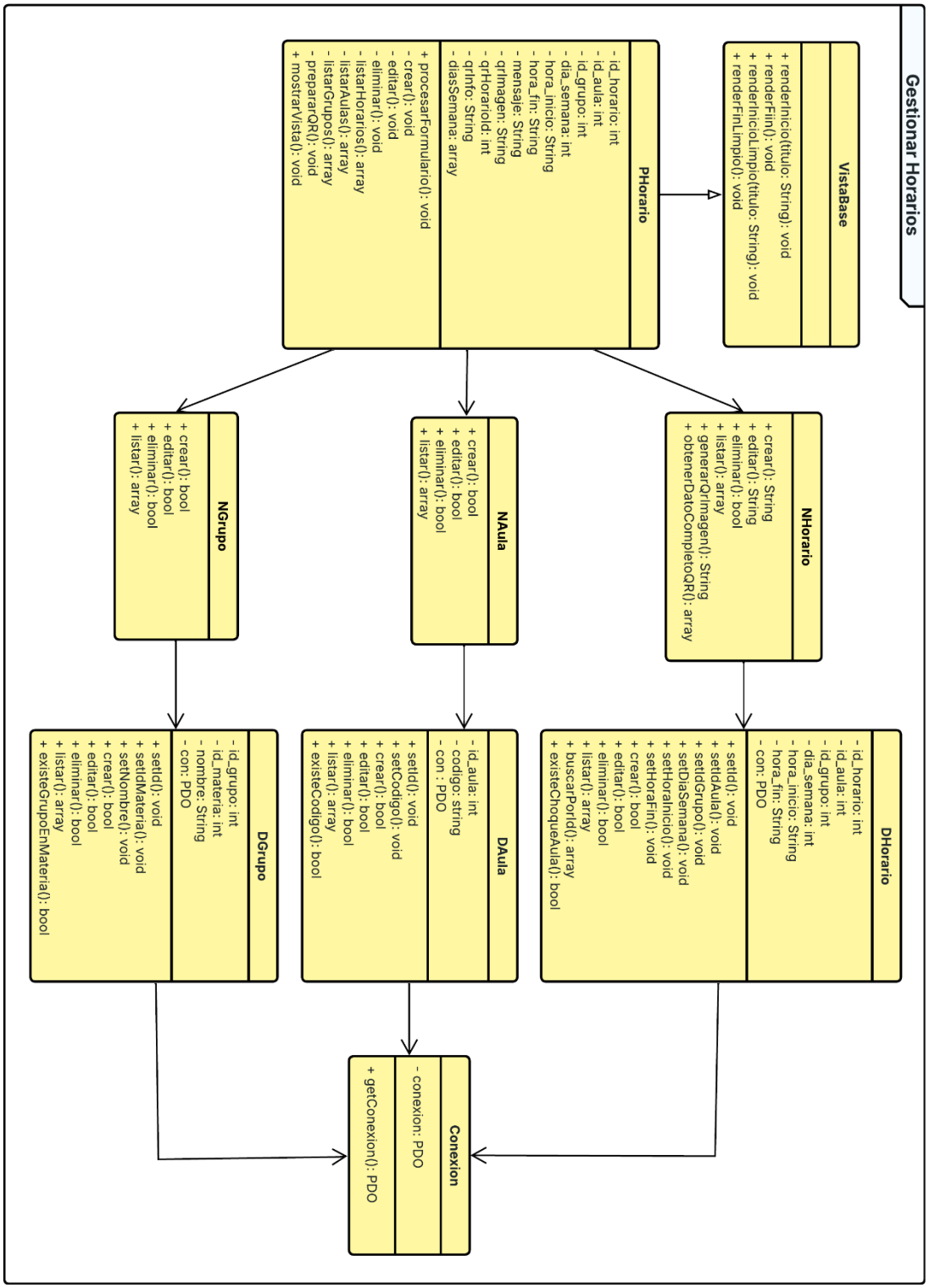
3.3.1.2 CU02: Gestionar Materias



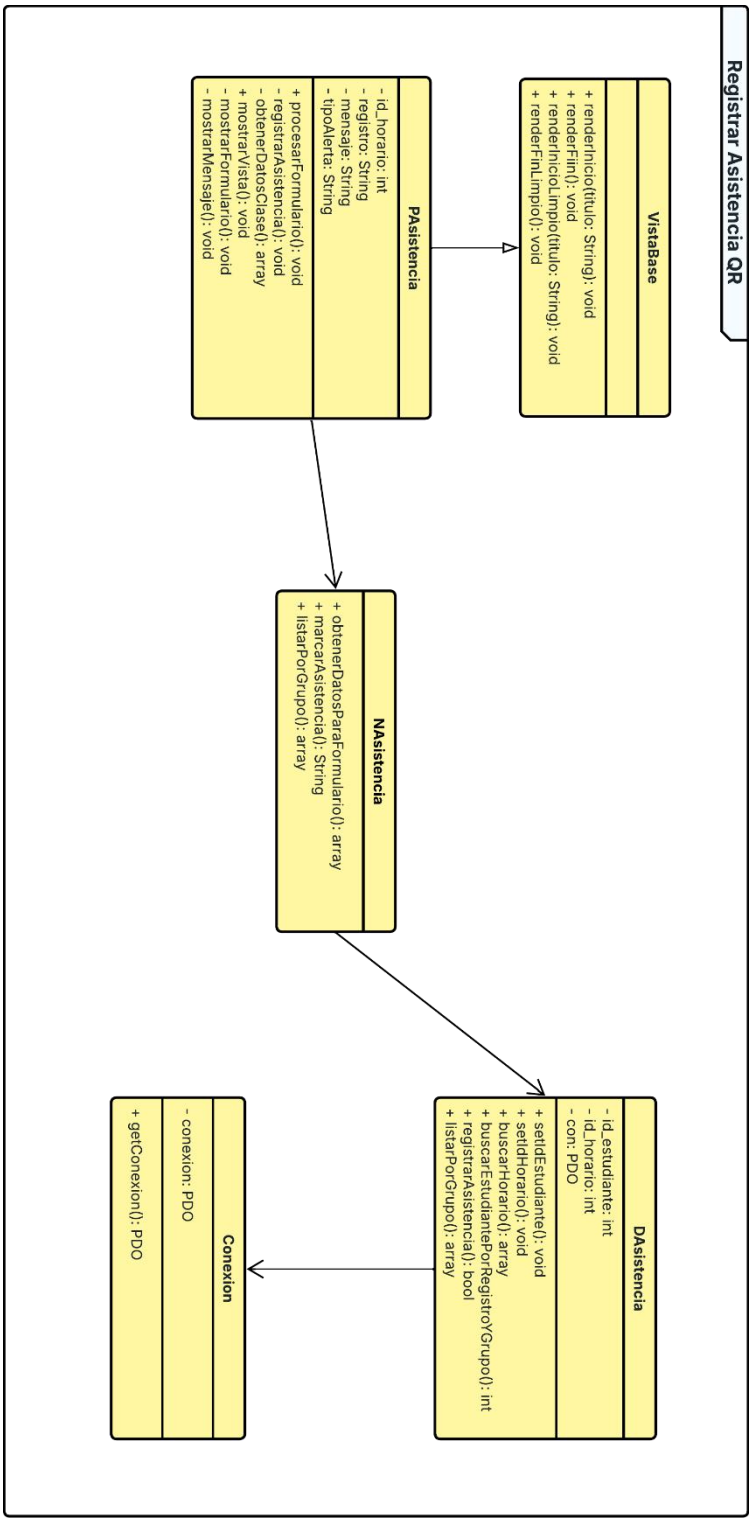
3.3.1.3 CU03: Gestionar Grupo



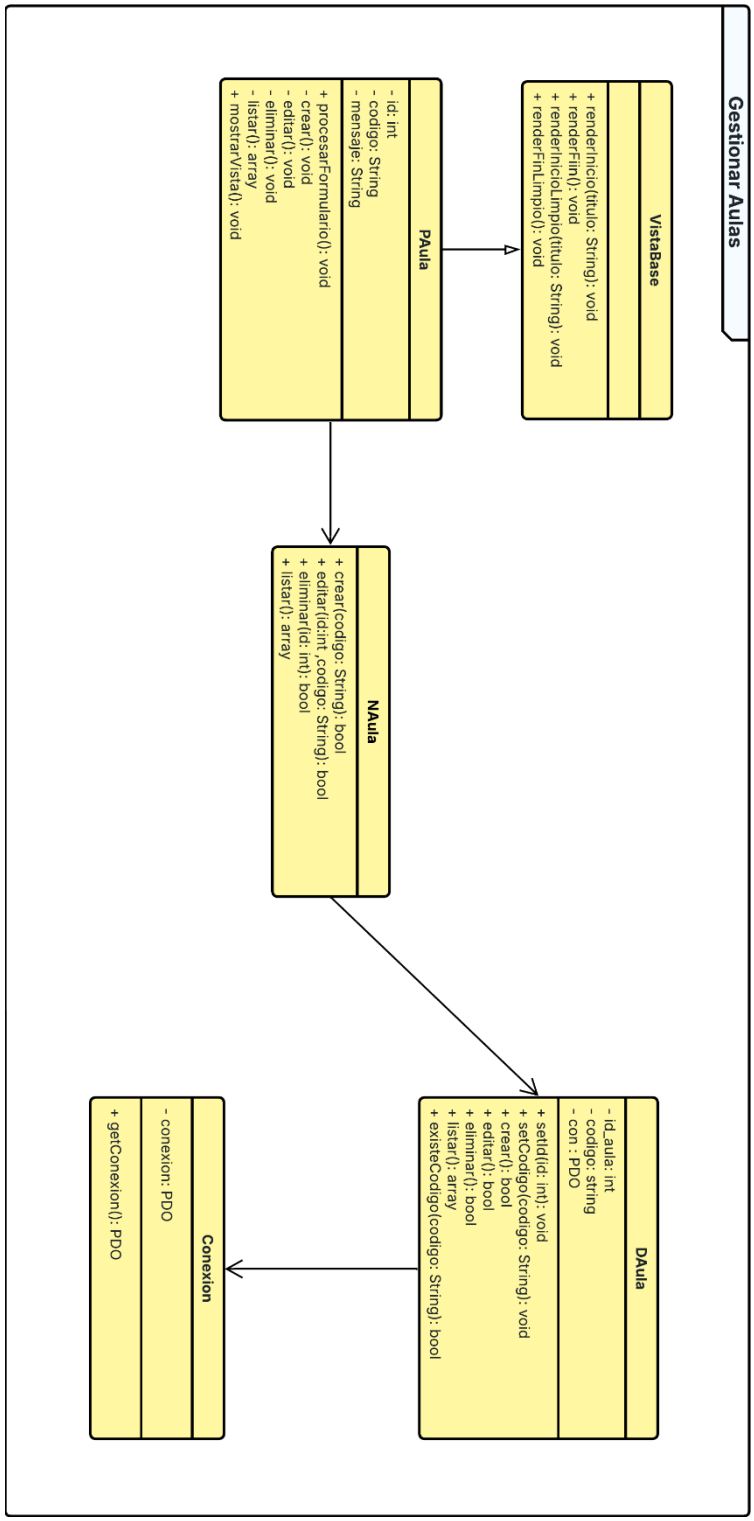
3.3.1.1 CU04: Gestionar Horario



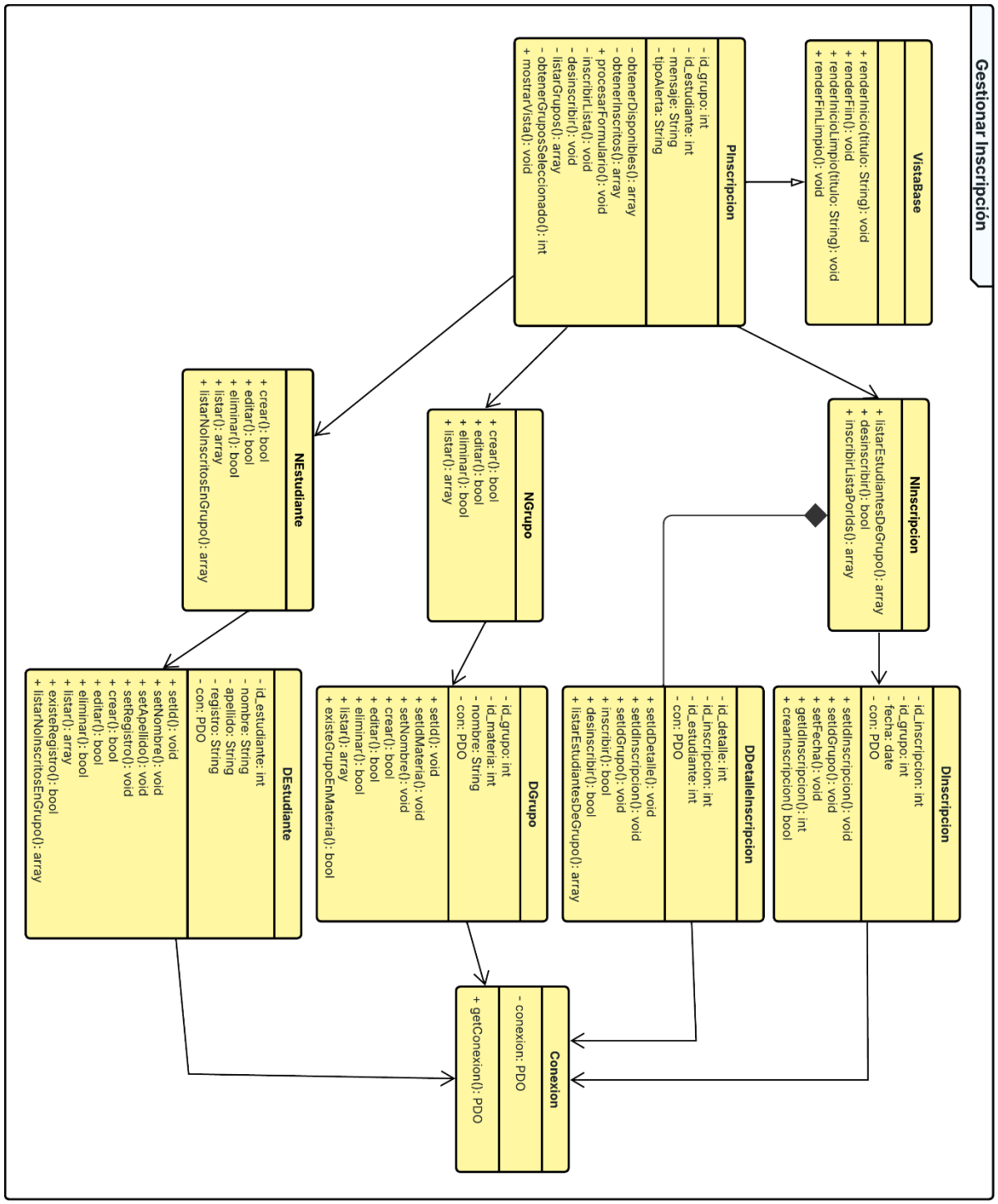
3.3.1.1 CU05: Registrar Asistencia QR



3.3.1.1 CU06: Gestionar Aula

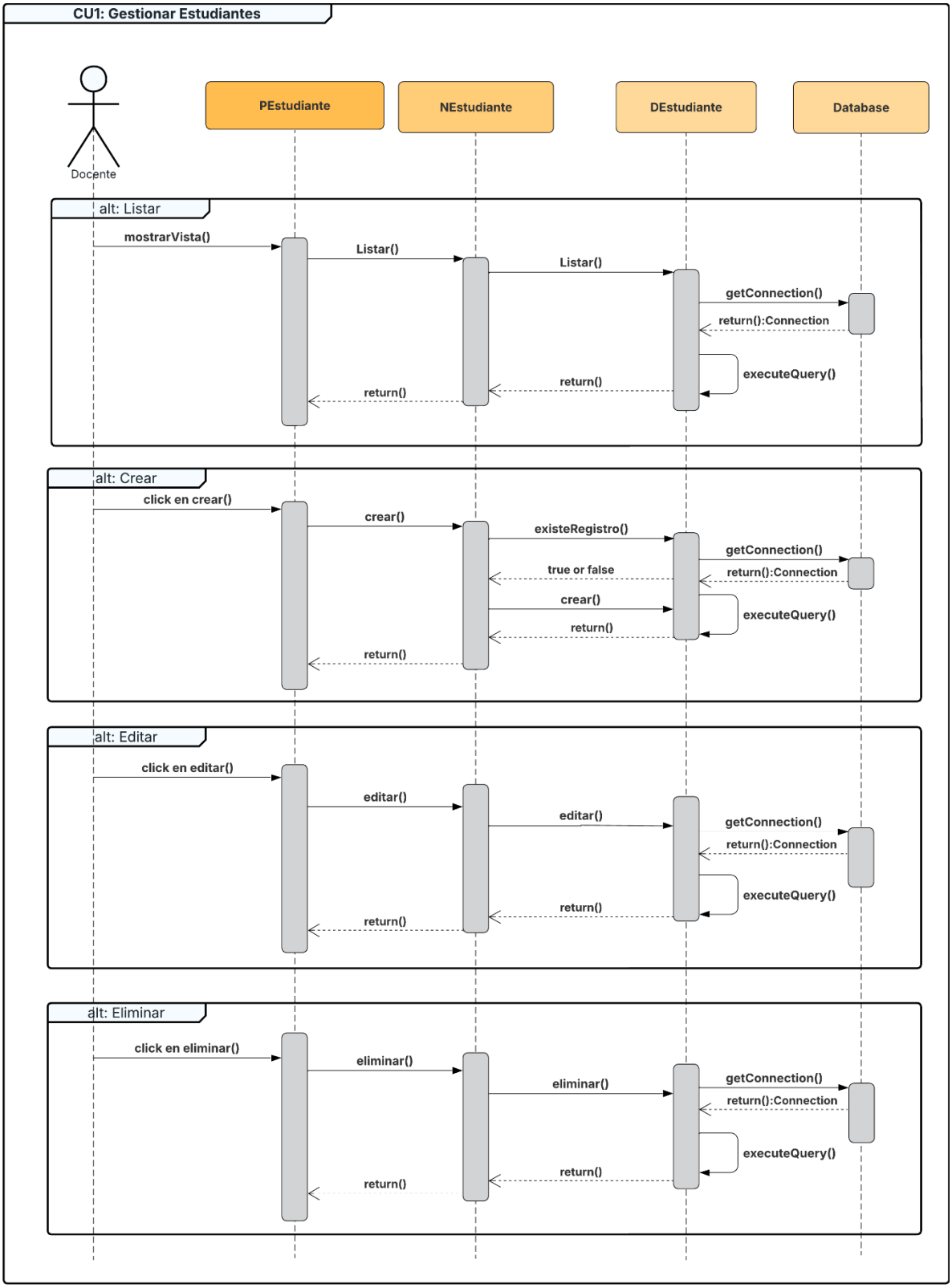


3.3.1.1 CU07: Gestionar Inscripción

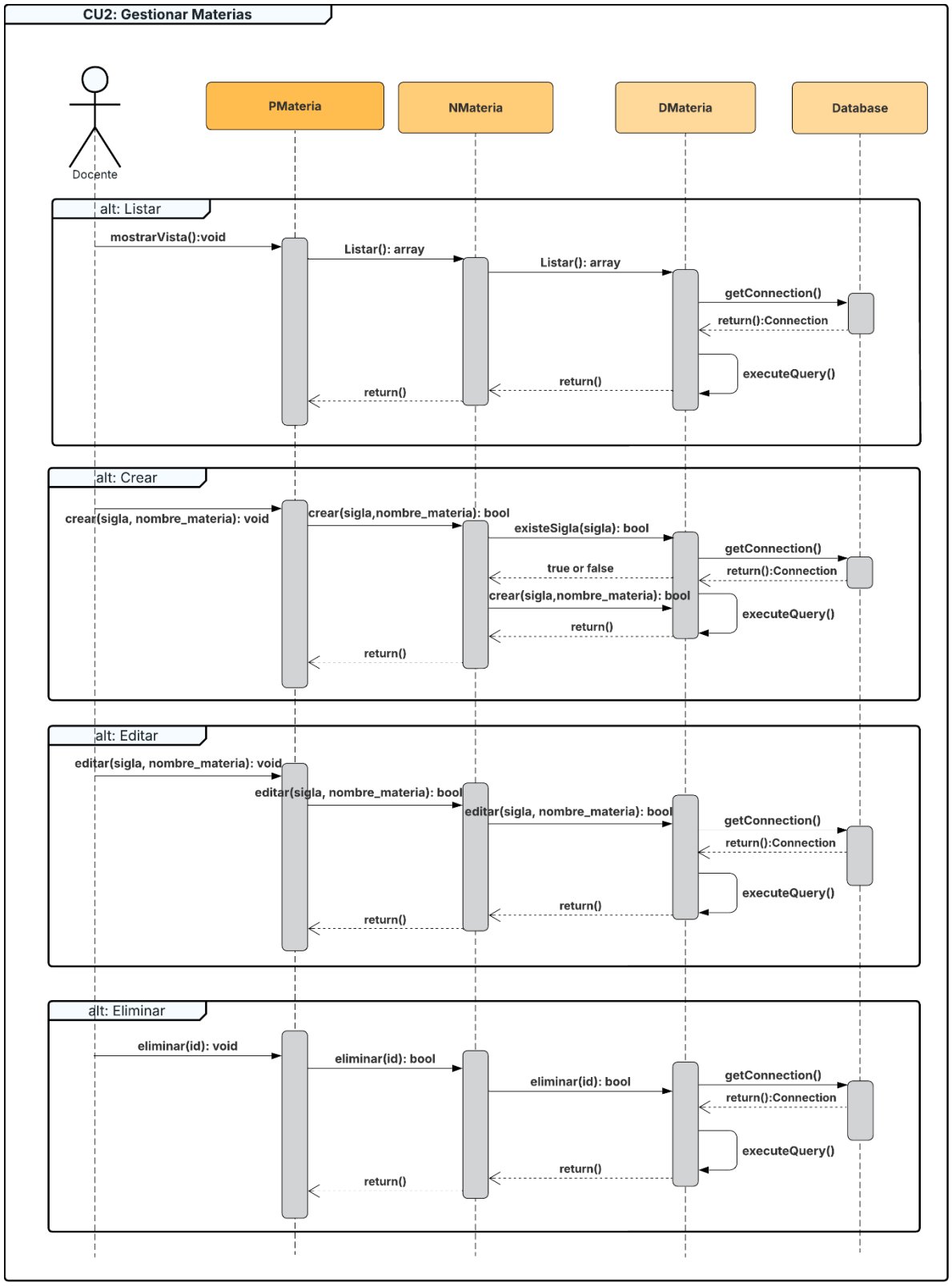


3.3.1 Diagrama de secuencia

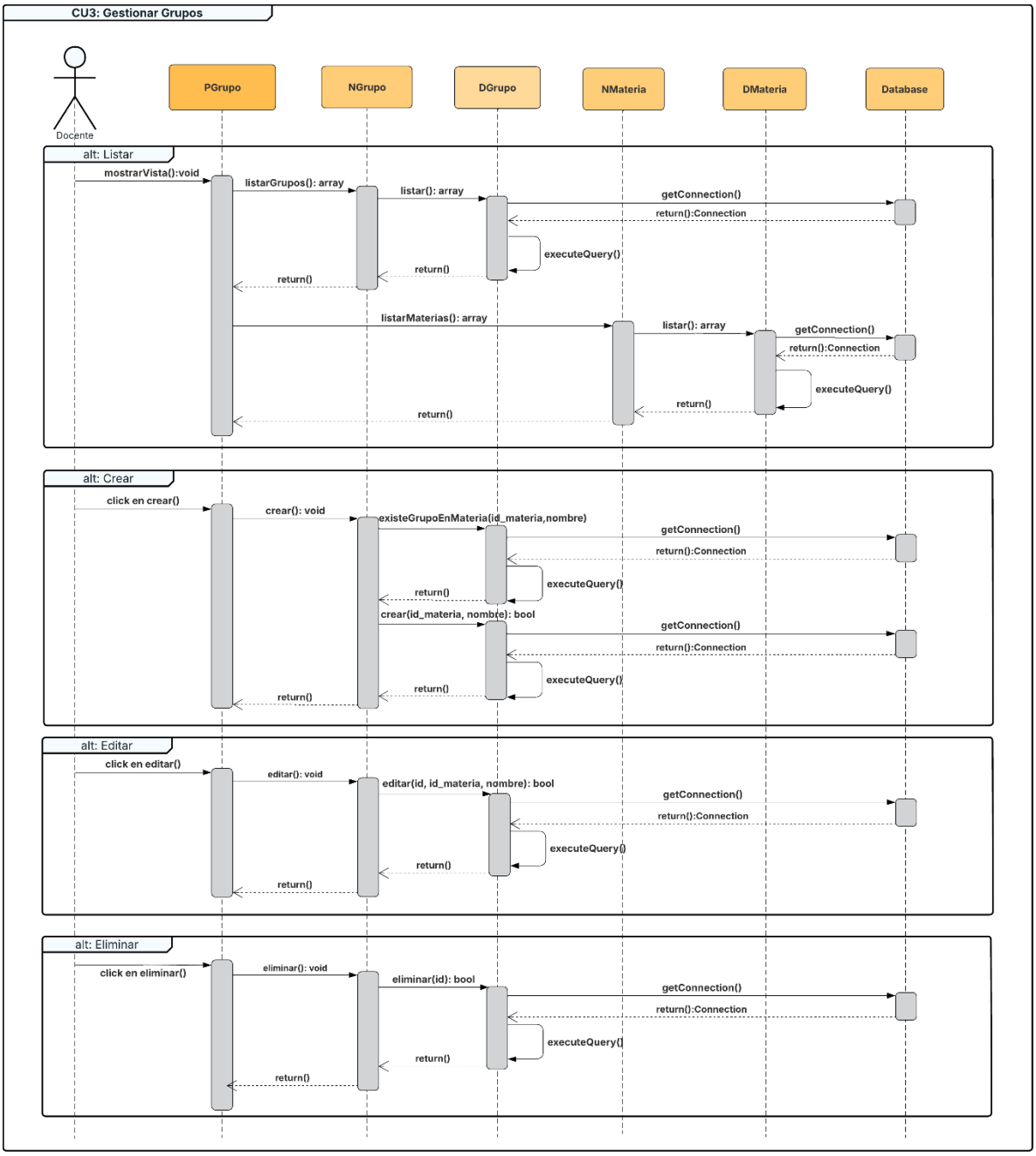
3.3.1.1 CU01: Gestionar Estudiantes



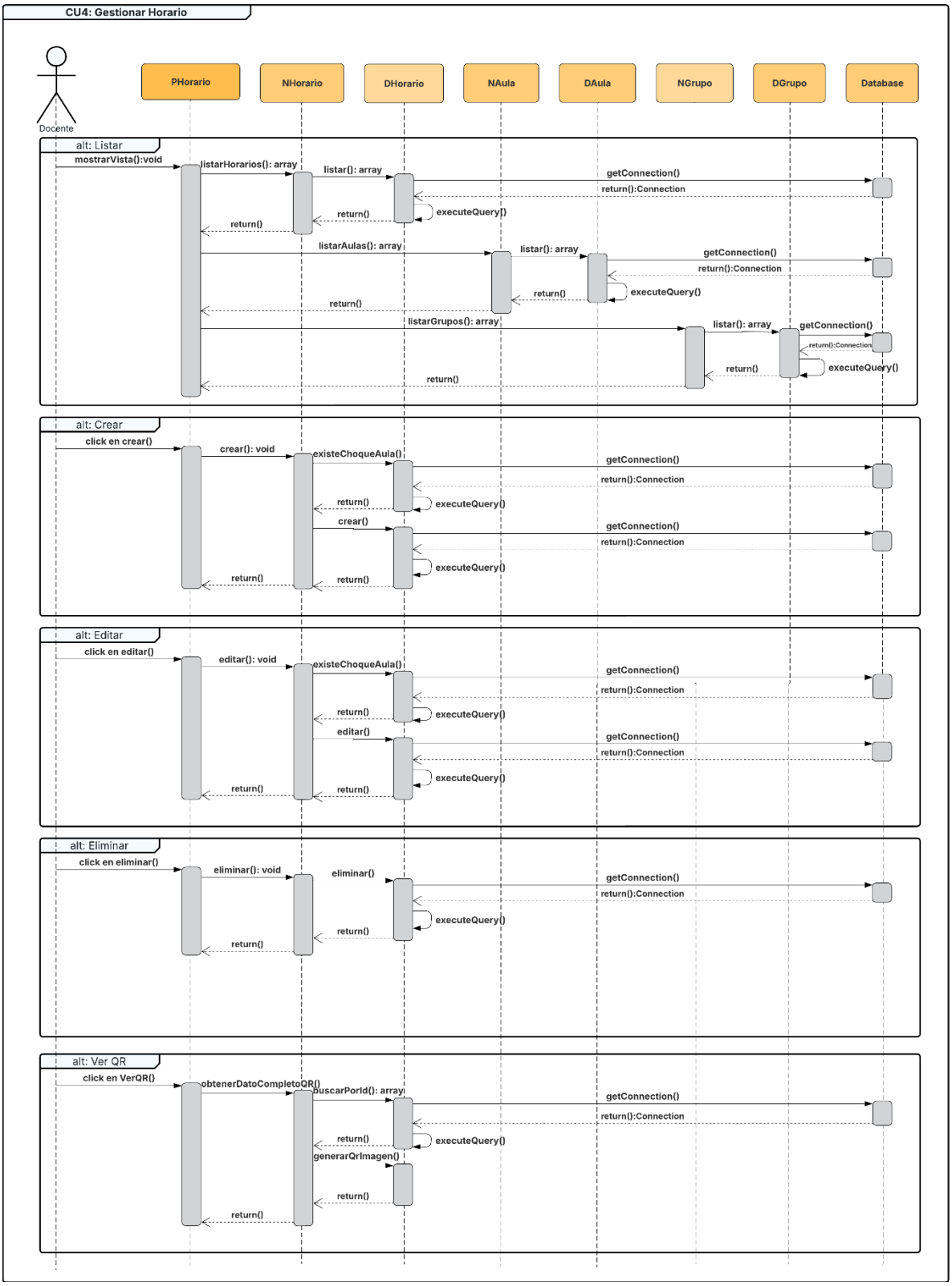
3.3.1.2 CU02: Gestionar Materias



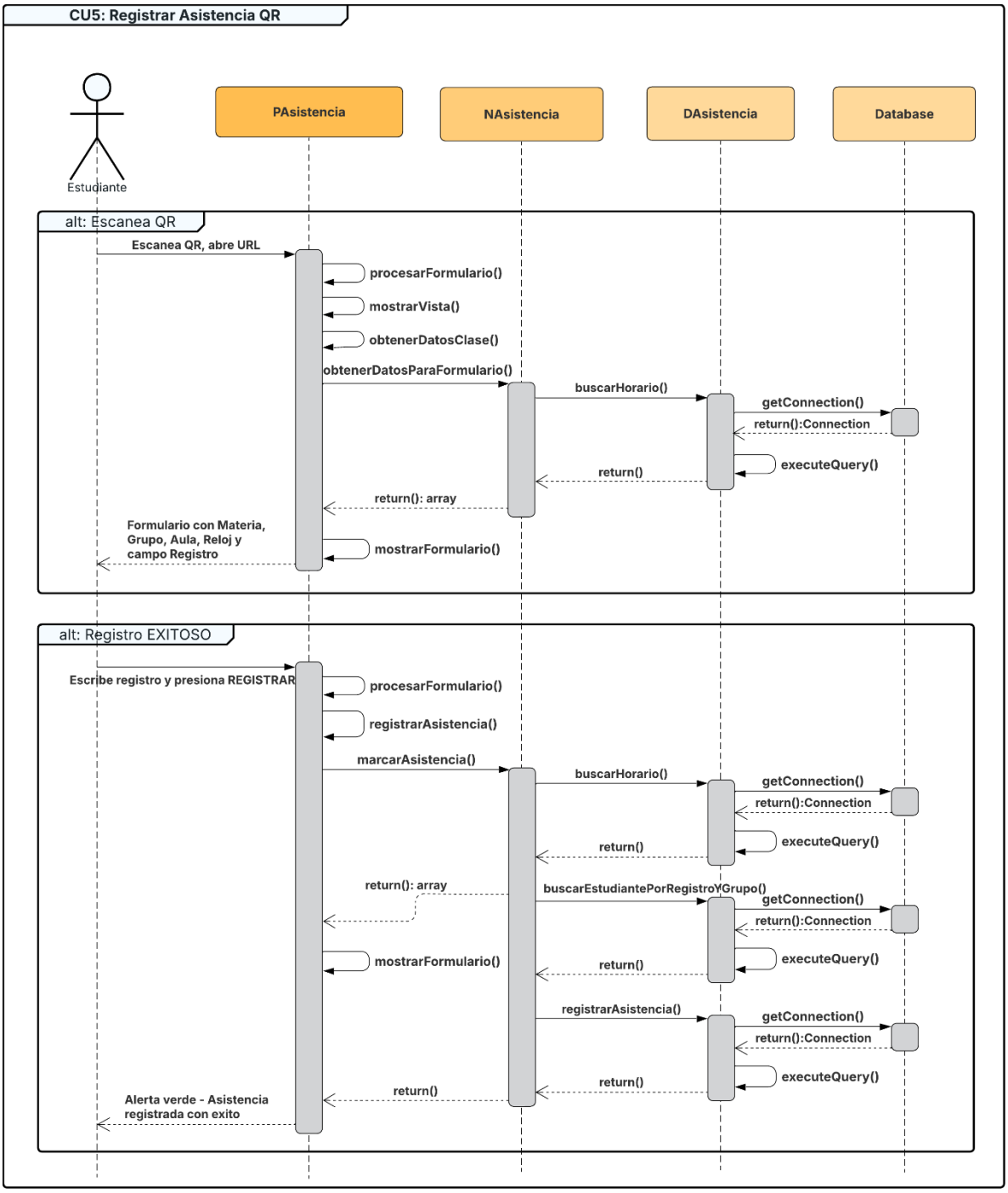
3.3.1.3 CU03: Gestionar Grupo



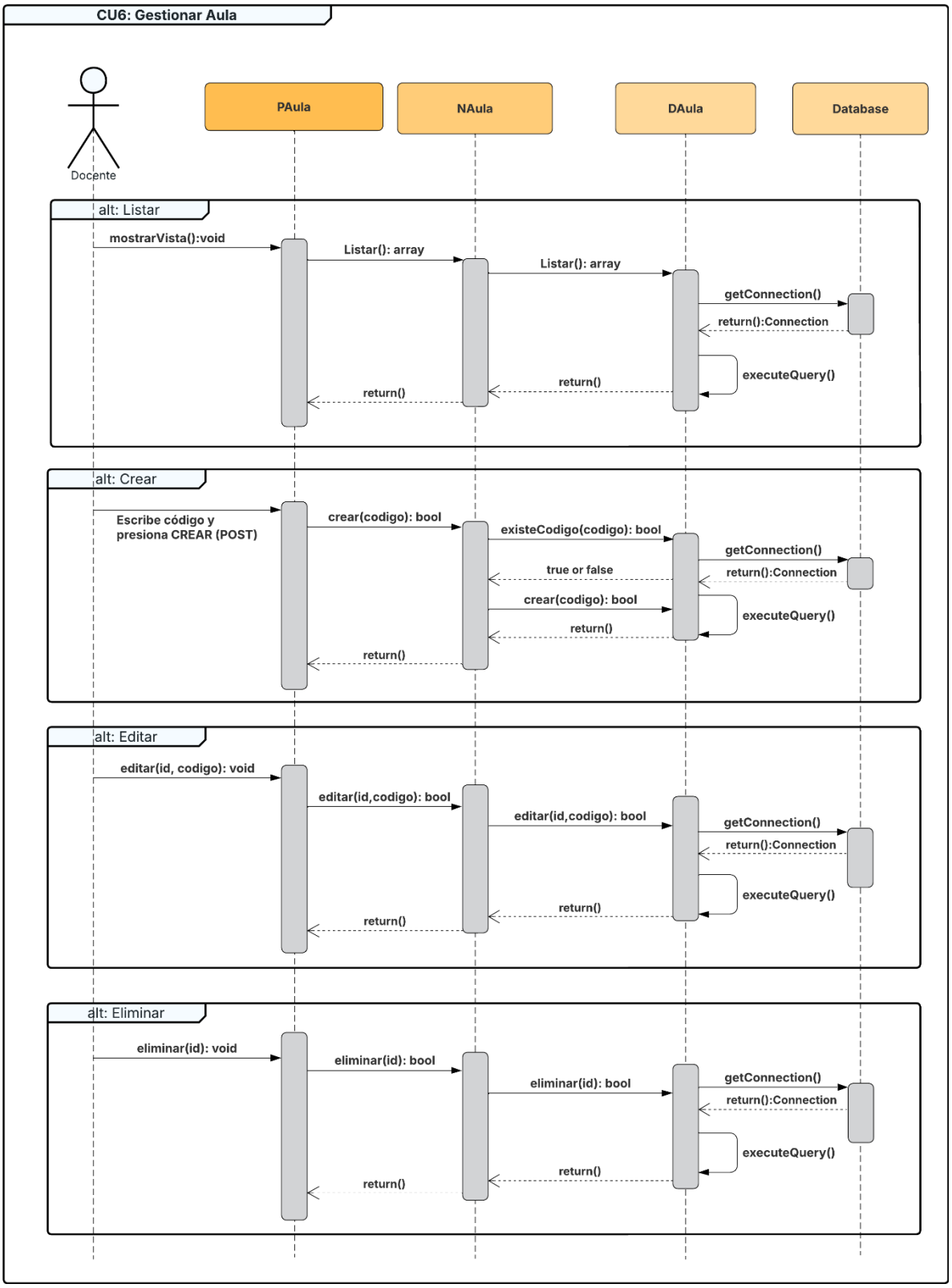
3.3.1.4 CU04: Gestionar Horario



3.3.1.5 CU05: Registrar Asistencia QR

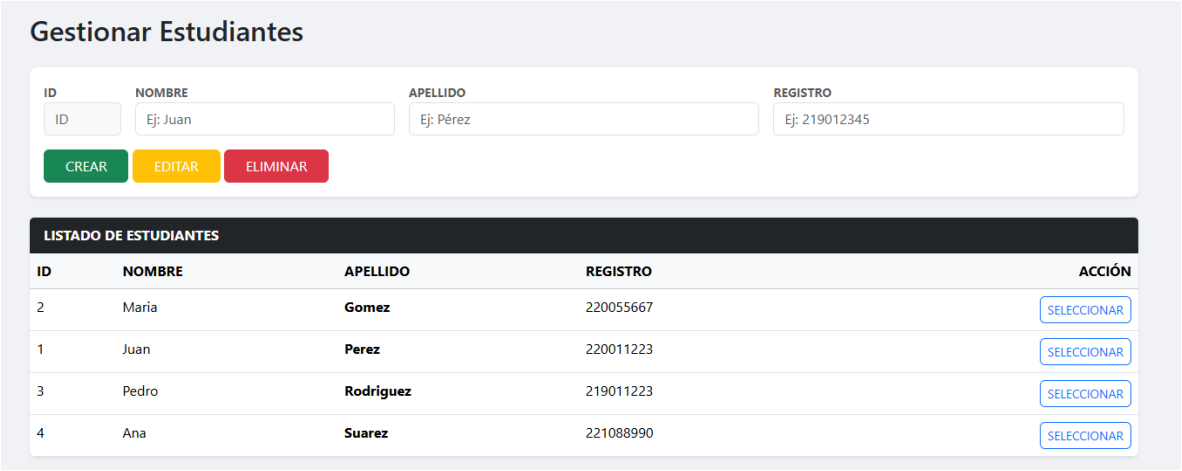


3.3.1.6 CU06: Gestionar Aula

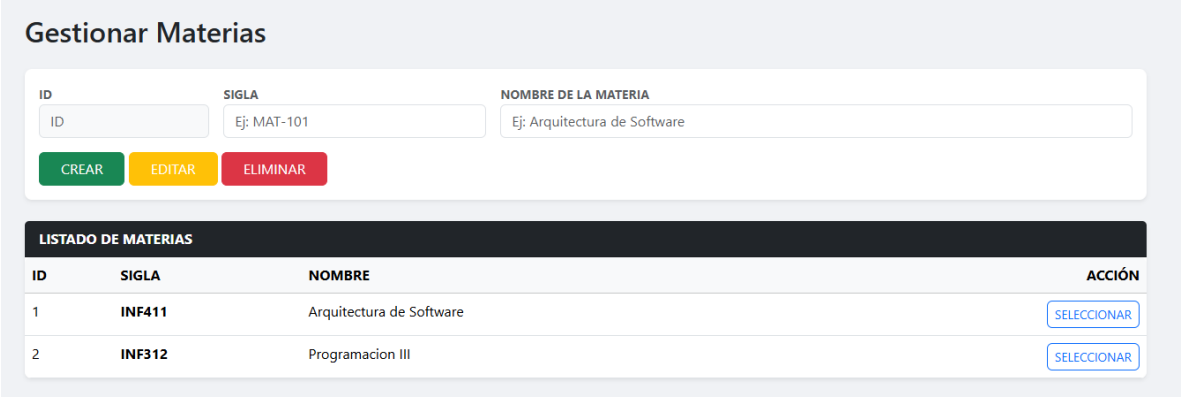


3.4 Diseño de la interfaz

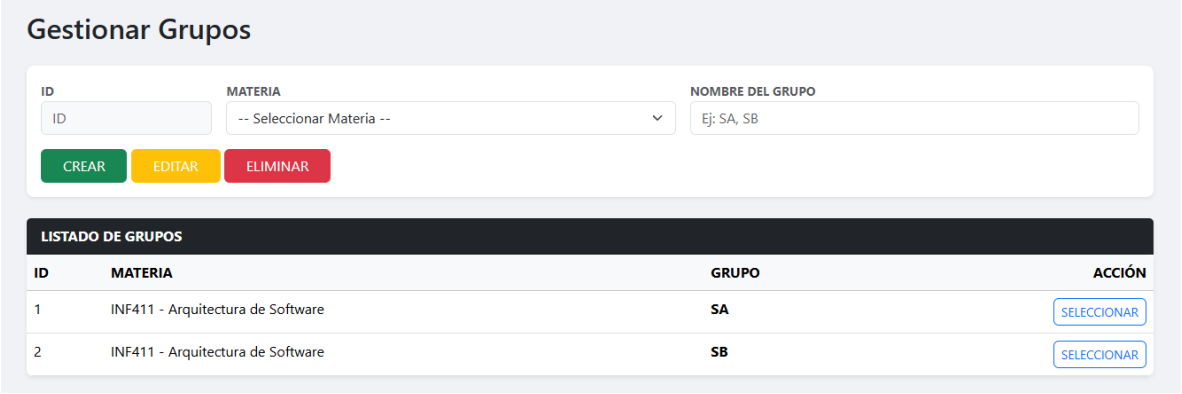
3.4.1 CU01: Gestionar Estudiantes



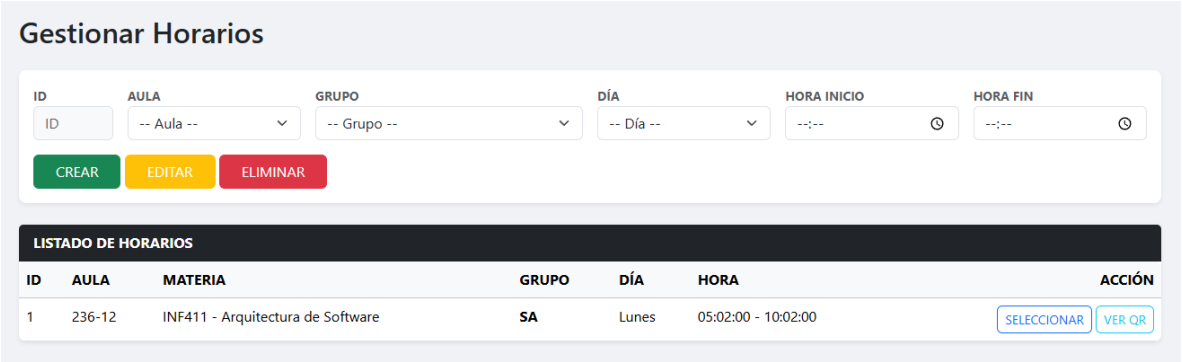
3.4.2 CU02: Gestionar Materias



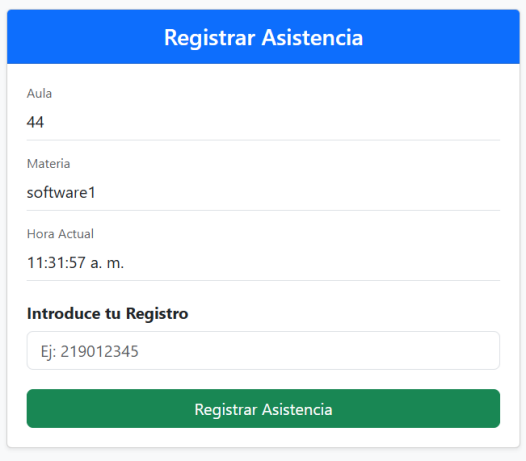
3.4.3 CU03: Gestionar Grupo



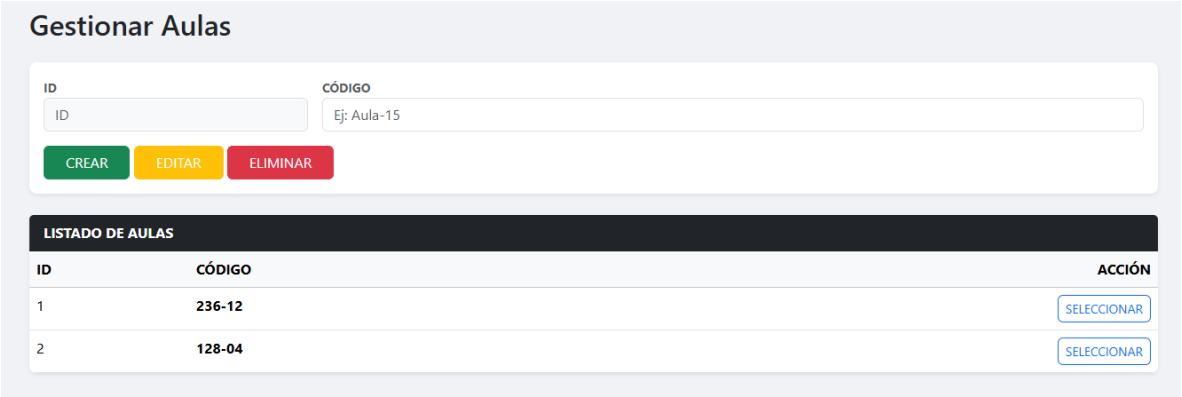
3.4.4 CU04: Gestionar Horario



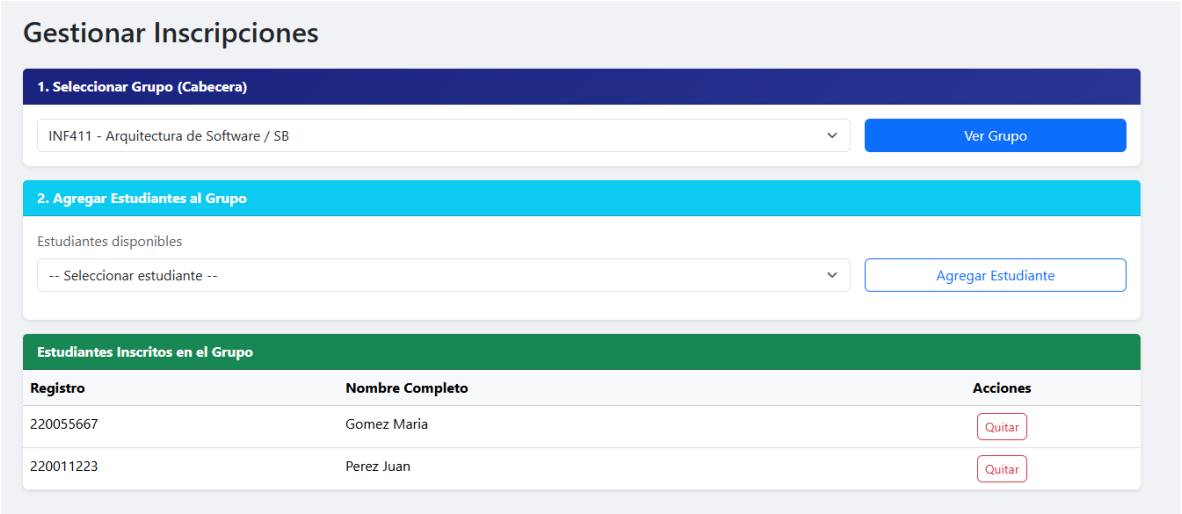
3.4.5 CU05: Registrar Asistencia QR



3.4.6 CU06: Gestionar Aula



3.4.7 CU07: Gestionar Inscripción



4 Bibliografia

1. JetBrains. (n.d.). *Compose Multiplatform Documentation*. Retrieved from https://www.jetbrains.com/compose/

2. Bootstrap. (n.d.). Bootstrap Documentation. Retrieved from https://getbootstrap.com/docs/

3. The PHP Group. (n.d.). PHP Manual. Retrieved from https://www.php.net/manual/en/

4. Microsoft. (n.d.). Visual Studio Code Documentation. Retrieved from https://code.visualstudio.com/docs

5. The PostgreSQL Global Development Group. (n.d.). PostgreSQL Documentation. Retrieved from https://www.postgresql.org/docs/